LatentSQL: Internalising a Multi-Agent Text-to-SQL Pipeline into the Latent Space of a Single Small Language Model

A step-by-step derivation, from first principles to the code

Abstract

Large language models solve Text-to-SQL well but are expensive to run. Small language models (SLMs) are cheap, but individually lack the reasoning capacity for hard queries. The standard remedy is to decompose the task across a *multi-agent system* — retrieval, schema selection, candidate generation and testing — which restores accuracy at the cost of many model calls, tens of thousands of generated tokens, extra training and coordination machinery, and error propagation between stages.

LatentSQL removes the internal coordination layer without pretending that database tools can be absorbed into a hidden state. Deterministic value retrieval remains an external operation and its hits are inserted once in the prompt at training and inference. One SLM then internalises exactly two model-side roles — schema linking and candidate generation — as *continuous thoughts*: hidden-state vectors fed back as input embeddings, emitting no intermediate tokens, before only the final SQL is decoded. Training retains only first-pass CHESS candidates with ordinary execution accuracy EX = 1. The candidate-generation supervision also contains training-only outcomes from deterministic one-edit negative SQL mutants, teaching the student to anticipate syntax errors, runtime errors, empty results and result mismatches without claiming access to execution feedback at inference. The architecture follows **IMAD** (Yi, Mueller and Lee, ACL 2026, "Latent Agents: A Post-Training Procedure for Internalized Multi-Agent Debate"), the latent mechanism follows **Coconut** (Hao et al., "Training Large Language Models to Reason in a Continuous Latent Space"), and the two internal roles are adapted from **CHESS** (Talaei et al., 2024).

This document explains that system from the very beginning, assuming no prior knowledge, and is organised strictly from the lowest level to the highest.

Part I states the problem from nothing. Part II descends to the smallest object in the system — one token, one embedding, one hidden state — and shows exactly where text is created. Part III proves, with the Data Processing Inequality and an entropy ceiling, that writing reasoning out as text is a strictly lossy channel, and computes the size of the loss with this project's own numbers. It then reduces bounded Text-to-SQL derivation to graph reachability and proves, under explicit representational assumptions, how a continuous state can perform parallel breadth-first search over schema, plan and query hypotheses. Part IV builds the continuous thought, the alignment map, and the latent working memory. Part V separates external retrieval from the two-role latent chain. Part VI gives the activation-space theory that makes this possible at all, including a proof that difference-in-means is the optimal linear probe under isotropic noise. Part VII derives the training loss exactly as the code computes it. Part VIII proves a small, uncomfortable fact about Adam that explains a failure in the earlier six-role curriculum. Part IX records that historical failure and its repair without presenting it as evidence for the new two-role model. Part X adds reinforcement learning. Part XI asks what that reinforcement stage can do to the latent geometry. Part XII maps every equation to its implementation. Part XIII separates completed legacy measurements from the current binary-versus-spurious experiment. Part XIV lists what is not established. Part XV collects every formula in one place.

Every symbol is defined before it is used. Every proof is given in full. Every number reported is one this project actually measured, and where two explanations are consistent with a measurement, both are stated.

Contents

I The problem, stated from nothing **7**

1 What a Text-to-SQL system does **7**
1.1 How we decide whether the output is right 7
1.2 The two benchmarks 8

2 Why one small model is not enough **8**

3 What a multi-agent pipeline does, and what it costs **8**

4 The idea of this project **9**

5 The formal problem **9**

II The lowest level: one token, one hidden state **10**

6 Step 1: the token and its embedding **10**

7 Step 2: the transformer as a function on embeddings **10**

8 Step 3: the language-model head — the only place text is created **11**

9 Step 4: what generating text actually costs **11**

III The decode bottleneck, made precise **12**

10 Three ways a reasoning step can hand off to the next **12**

11 Background: entropy and mutual information **12**

12 The setup, matching what the code and the baselines actually do **12**

13 Theorem: text handoff cannot beat its own token count **13**
13.1 Putting our numbers in 14
13.2 What this proves, and — importantly — what it does not 14

14 The second, different argument: superposition and search **15**
14.1 What the papers actually compare 15
14.2 Why Text-to-SQL becomes a search graph 16
14.3 Deriving breadth-first search as vector algebra 18
14.4 How two transformer layers implement the update 19
14.5 The conditional Text-to-SQL theorem 20

14.6 Why ordinary decoded CoT does not get this property for free 22
14.7 Why gradient training can produce the mixture 22
14.8 A separate guarantee: reward latent SQL trajectories by execution 24
14.9 What the three proofs jointly establish—and what they do not 25

IV The continuous thought **26**

15 Step 1: latent mode **26**

16 Step 2: the alignment map **26**
16.1 Construction 1: identity (Coconut's own choice) 26
16.2 Construction 2: linear 27
16.3 Construction 3: soft embedding (the default) 27

17 Step 3: latent working memory (the KV cache) **30**
17.1 A subtlety the code had to solve: cache mutation under checkpointing 30

18 Step 4: the cost accounting **31**

19 Is this beam search? Is it tree search? A precise comparison **31**
19.1 Before any formal definition: the exact same numbers, two ways 31

V From a multi-agent pipeline to a chain of latent slots **34**

20 What remains outside the student **34**

21 How the SFT records are selected **34**

22 The two-role chain **34**

23 The tag vocabulary and sequence **35**

24 The curriculum: stage k **35**

VI Activation space: why any of this can work **37**

25 The residual stream and the linear representation hypothesis **37**

26 Difference-in-means, and a proof that it is the right direction **37**

27 Steering: using the direction **38**

28 IMAD's finding, and what it licenses **39**

29 What this does and does not establish for LatentSQL **39**

VII The training objective **40**

30 Cross-entropy, from scratch **40**

31 The COCONUT sequence loss, exactly as the code computes it **40**
31.1 Why the latent positions get gradients anyway 41

32 The optional SQL-readiness probe (disabled in the current run) **42**
32.1 A degeneracy: why $j^\star = k$ does nothing 42
32.2 Why $j^\star = k - 1$ is not degenerate 43

VIII The optimiser, and one uncomfortable fact about Adam **44**

33 What is actually being trained: LoRA **44**

34 Adam, derived **44**
34.1 The normalisation, and what it costs 44

35 Gradient accumulation, and how few steps there really are **45**

IX A historical worked failure: the obsolete six-role stage-4 collapse **47**

36 The symptom **47**

37 What the failures actually looked like **47**

38 Mechanism 1: the objective is saturated by stage 3 **48**

39 Mechanism 2: Adam steps into the noise at full size **48**

40 Mechanism 3: where the drift lands **49**

41 The bonus finding: the fix that was already configured did nothing **49**

42 The repair **49**

43 Result of the retrain **50**

44 Training loss during the retrain, and the attribution caveat **50**

45 Two infrastructure failures worth recording **51**

X Reinforcement learning: internalisation with GRPO **52**

46 Group Relative Policy Optimisation, from the policy-gradient theorem **52**

47 The clipped surrogate and the KL penalty **53**

48 IMAD's reward, specialised to SQL execution **53**
48.1 Dense partial credit, for when GRPO would otherwise learn nothing 53
48.2 Brevity, for when a group is entirely correct 54
48.3 Spurious correctness: when a group is entirely correct, but shouldn't be 54

49 Latent rollouts **56**

50 What GRPO measurably did in the legacy six-role run **56**

XI Does GRPO sharpen the agent subspaces, or not? **58**

51 The one fact everything else follows from **58**

52 A die you roll versus a die you average **58**
52.1 Turning that into a one-line proof 59

53 Two published fixes, and their actual formulas **60**
53.1 Fix 1 — roll inside the formula (Soft-GRPO / Latent-GRPO) 60
53.2 Fix 2 — roll the weights instead of the output (Dropout-GRPO) 60

54 Neither fix is in this project's code **61**

55 So why did our own run still move the numbers? **61**
55.1 How SQL reward still changes the shared latent trajectory 61

56 A separate question: did SFT even need to *create* the subspaces? **62**

57 Summary **63**

58 A sibling project, and a genuinely different fix to a shared problem: SERL-SQL **63**
58.1 The same coarse-credit problem, independently named 63
58.2 Two different fixes to the same complaint 64
58.3 A speculative bridge (unimplemented, stated as such) 64
58.4 The one architectural difference that is not open to interpretation 65

XII Complete map from equation to code **66**

59 The corrected end-to-end pipeline **67**

XIII Results, reported honestly, including where the method loses **68**

60 The two tests that actually define success **68**
60.1 An earlier, text-mode arm: Test A never passed 68

61 Test B on the earlier arm, before the architectural change **68**

62 Legacy six-role SQL-direct results **69**

63 Current two-role binary-versus-spurious experiment **69**

64 Cost **69**
64.1 Speculative decoding: measured, and a dead end on this path 69

65 Smoke-test coverage (verifies mechanism, not accuracy) **70**

XIV What is not established **71**

66 Required ablations, not yet run **71**

67 The historical attribution gap in the stage-4 repair **71**

68 What Part VI does not establish for this system **71**

69 What Part XI does not establish **71**

70 What Part III's capacity argument does not establish **72**

71 What Part X does not establish **72**

72 Infrastructure and scale limits **72**

XV Every formula in one place **73**

73 The one-paragraph version **74**

Part I
The problem, stated from nothing

1 What a Text-to-SQL system does

A **database** is a collection of tables. A **schema** is the description of those tables: their names, their columns, and the type of each column. Here is a tiny one, written in the language databases themselves use:

```
CREATE TABLE museum   (museum_id INT, name TEXT, opened INT);
CREATE TABLE visitor  (visitor_id INT, name TEXT, age INT);
CREATE TABLE visit    (museum_id INT, visitor_id INT, spent REAL);
```

A **query** is a program in the language SQL that asks a question of those tables and returns a small table of numbers or strings as its answer:

```
SELECT name FROM visitor WHERE age > 30;
```

Text-to-SQL is the task of writing that program automatically. The input is a question in English together with the schema; the output is a SQL query. Formally, we want a function

$$f : (q, S) \longmapsto y,$$

where q is the question, S is the schema, and y is a SQL query.

1.1 How we decide whether the output is right

We do *not* compare the generated query to the gold query as strings. Two different SQL programs very often compute exactly the same answer, and a string comparison would call one of them wrong for no reason.

Definition 1 (Execution accuracy, EX)**.** Let y be the predicted query and $y^\star$ the gold query, both run against the same database D. Write $\mathrm{exec}(y, D)$ for the table of results returned. Then the example is scored

$$\mathrm{EX}(y, y^\star, D) = \begin{cases} 1 & \text{if } \mathrm{exec}(y, D) = \mathrm{exec}(y^\star, D), \\ 0 & \text{otherwise (including if } y \text{ fails to parse or errors).} \end{cases}$$

Over a test set of n examples, **execution accuracy** is the mean of this quantity, reported as a percentage.

Remark 2. Comparison of two result tables needs one subtlety: if the gold query contains an `ORDER BY` clause, row order is part of the answer and must match; if it does not, the two tables are compared as multisets of rows. This is what `latentsql/db/execute.py` does, via `gold_is_ordered` and `result_match`.

> Every accuracy number in this document is EX in the sense of Definition 1: the query was actually executed against the actual database, and the returned rows were compared. Nothing is scored by string similarity.

1.2 The two benchmarks

We use a 50/50 mixture of two standard datasets.

- **Spider**: many small databases, questions that are mostly single-table or simple joins. Easier.
- **BIRD**: large, messy, real-world databases, questions that need external "evidence" hints and often numeric reasoning. Much harder.

Because the two differ so much in difficulty, a single averaged number hides what happened. Every result table below therefore reports EX overall *and* EX on each half separately.

2 Why one small model is not enough

A **language model** is a function that, given a sequence of text, produces a probability distribution over the next token; generating text means sampling from that distribution repeatedly. A **small language model (SLM)** here means one with roughly 7 billion parameters or fewer — specifically `Qwen2.5-Coder-7B-Instruct`, which fits on a single 24 GB consumer GPU.

Ask such a model a hard BIRD question directly, in one shot, and it frequently fails. The reason is not that it lacks knowledge of SQL. A complete deployed system must perform several distinct kinds of work:

1. Find which literal values in the question actually occur in the database ("Los Angeles" might be stored as `LA`).
2. Decide which of 80 tables and 600 columns are relevant at all.
3. Decide whether the query is simple, needs a join, or needs a nested subquery.
4. Write a candidate query.
5. Run it, see the error, and fix it.
6. Check that the fixed query actually answers the question asked.

These six operations describe the full system, not six things that should all be hidden inside the student. Retrieval requires a database or value index and therefore remains external. Execution-based revision and verification require an executor and are supplied later as training signals, not as fictional inference-time latent tools. The internalised student is responsible for the two model-side operations that remain: schema linking and candidate generation.

3 What a multi-agent pipeline does, and what it costs

The established fix is to give the model scratch space by *decomposing* the task. Instead of one call, a pipeline makes several calls, each with its own prompt and tool access, and passes written output onward. DIN-SQL, DAIL-SQL, CHESS and CHASE-SQL arrange these operations differently. CHESS names four specialised agents — Information Retriever, Schema Selector, Candidate Generator and Unit Tester. LatentSQL does not rename DIN-SQL's classifier or CHESS's revise operation as additional CHESS agents: it keeps retrieval outside the student, internalises CHESS's schema-selection and candidate-generation work, and leaves execution-based correction to offline supervision and GRPO.

The multi-agent system works. It is not a straw man. It genuinely raises accuracy, and it is the right baseline to beat.

But it costs four separate things:

1. **Tokens.** Every intermediate result must be *written out* in words so the next stage can read it. In this project's own measurements, a fully verbalised run averages 2922.8 total tokens per question, of which 1339.3 are generated.
2. **Latency and money.** Each stage is a separate forward pass over a long prompt.
3. **Machinery.** Someone must write, tune, and maintain the orchestration: the prompts, the routing, the retry logic.
4. **Error propagation.** If schema linking drops the one column the answer needs, no later stage can recover it. The mistake is now baked into the text that everything downstream reads.

4 The idea of this project

Keep the decomposition. Delete the text.

Train one small model so that each pipeline stage happens as a handful of *internal vector states* rather than as written words, and only the final SQL query is ever decoded into tokens.

If this works, all four costs fall at once: the intermediate tokens disappear (they were never generated), the extra forward passes collapse into one sequence, the orchestration is gone (it is inside the weights), and error propagation is softened because a vector can carry an uncertain, partially formed hypothesis in a way a committed sentence cannot — a point made precise in Section 14.

5 The formal problem

Definition 3 (The object we are building)**.** Let π_θ be a language model with parameters θ. We want to choose θ so that, on a held-out set $\{(q_i, S_i, y_i^\star, D_i)\}_{i=1}^n$, the model's output y_i maximises

$$\frac{1}{n}\sum_{i=1}^{n} \mathrm{EX}(y_i, y_i^\star, D_i)$$

subject to the constraint that the number of tokens decoded per question is small — ideally just the SQL query itself, roughly 60 tokens.

Note carefully what is *not* the goal. We are not trying to beat a frontier LLM. We are trying to answer two much sharper questions:

Test A. Does a single SLM that runs the pipeline *latently* beat the same SLM trained to run it *in text*?

Test B. Does it match the explicit multi-agent system at the same model size, at a fraction of the tokens?

Everything below is (i) building the latent mechanism, (ii) proving what it can and cannot buy, (iii) training it, and (iv) measuring it.

Part II
The lowest level: one token, one hidden state

Nothing in this part is specific to our method. It is the standard machinery of a transformer language model, written out in the exact notation the rest of the document uses. If you already know it, you can skim to Section 9, which is where the thesis actually begins.

6 Step 1: the token and its embedding

Text is first cut into **tokens** — pieces roughly the size of a common word or a fragment of a rare one. The set of all possible tokens is the **vocabulary** V; each token is identified by an integer index. For our model,

$$|V| = 152{,}064.$$

A model cannot do arithmetic on an integer index, so each token x is mapped to a vector by an **embedding function**

$$e : V \to \mathbb{R}^d, \qquad d = 3584 \text{ for our model.}$$

Concretely e is a lookup in a matrix $W_{\text{in}} \in \mathbb{R}^{|V| \times d}$: row x of that matrix *is* $e(x)$. The number d is called the **hidden dimension**.

Example 4. If the token `"SELECT"` has index 4416, then $e(\texttt{"SELECT"})$ is row 4416 of W_{in}: a specific list of 3584 real numbers.

> **The single most important fact in this document:** the model does not actually consume tokens. It consumes *vectors in* $\mathbb{R}^d$. Tokens are merely the usual way of producing those vectors. Nothing forces the input vector at a position to be a row of W_{in}.

7 Step 2: the transformer as a function on embeddings

Given a sequence of tokens $x = (x_1, \ldots, x_T)$, form the matrix of their embeddings up to position t:

$$E_t = \big[\, e(x_1),\ e(x_2),\ \ldots,\ e(x_t) \,\big] \in \mathbb{R}^{t \times d}.$$

The transformer is a function that maps this to a matrix of the same shape:

$$H_t = \text{Transformer}(E_t) \in \mathbb{R}^{t \times d}. \tag{2.1}$$

Row s of H_t is the model's internal representation of position s after it has looked at positions 1 through s. We write

$$h_t = H_t[t, :] \in \mathbb{R}^d$$

for the last row: the **last hidden state**, the representation of the most recent position. This is the vector the whole method is built on.

Remark 5 (causality)**.** The transformer is *causal*: row s depends only on positions $\leq s$. That is what makes it possible to compute $h_1, h_2, \ldots$ one at a time and reuse the earlier work — see Section 17.

Remark 6 (the final normalisation)**.** h_t as we use it is taken *after* the model's final normalisation layer, so its magnitude is controlled — a typical entry is order 1, not order 100. Coconut makes exactly this observation, and it matters: a raw, un-normalised state fed back as an input would be far outside the range the model's first layer expects.

8 Step 3: the language-model head — the only place text is created

To turn h_t into a next token, the model applies one more matrix, the **output head** $W_{\text{out}} \in \mathbb{R}^{|V|\times d}$, and then a softmax:

$$\ell_t = W_{\text{out}}\, h_t \in \mathbb{R}^{|V|}, \qquad p(x_{t+1} = v \mid x_{\le t}) = \text{softmax}(\ell_t)_v = \frac{\exp(\ell_{t,v})}{\sum_{u\in V}\exp(\ell_{t,u})}. \tag{2.2}$$

The entries of ℓ_t are called **logits**. The softmax turns them into a probability distribution: every entry lands in $(0,1)$ and they sum to 1.

> Equation (2.2) is the *only* step in the entire model where a continuous internal state becomes discrete text. Everything before it is vectors.

9 Step 4: what generating text actually costs

Ordinary generation — what chain-of-thought does, and what a multi-agent pipeline does between its stages — is the following loop:

$$h_t \xrightarrow{W_{\text{out}},\ \text{softmax}} p(\cdot) \xrightarrow{\text{sample or argmax}} x_{t+1} \xrightarrow{e(\cdot)} e(x_{t+1}) = W_{\text{in}}[x_{t+1},:] \longrightarrow \text{next step's input.} \tag{2.3}$$

Look at the shapes along that path:

$$\underbrace{\mathbb{R}^{3584}}_{h_t} \longrightarrow \underbrace{\Delta^{|V|-1}}_{\text{a distribution}} \longrightarrow \underbrace{\{1,\dots,152064\}}_{\text{one integer}} \longrightarrow \underbrace{\mathbb{R}^{3584}}_{e(x_{t+1})}.$$

The middle arrow is the problem. A vector of 3584 real numbers is collapsed into a single integer chosen from 152,064 possibilities, and then re-expanded. Whatever the state was holding that was not expressible as "the identity of one token" is gone, permanently, and no later step can get it back.

Example 7 (the collapse, in miniature)**.** Take a toy model with $d = 3$ and a vocabulary of 4 tokens $V = \{A, B, C, D\}$. Suppose at some step the last hidden state produces logits

$$\ell = (2.0,\ 1.9,\ 0.1,\ 0.0) \quad\Longrightarrow\quad p \approx (0.44,\ 0.40,\ 0.07,\ 0.06).$$

The state is genuinely undecided between A and B: it holds *both* hypotheses, with almost equal weight. Greedy decoding emits A, and the next step's input becomes exactly $e(A)$. The distinction between "I am sure it is A" (say $p = (0.99, 0.003, \dots)$) and "it is probably A but might well be B" has been erased: both produce the identical next input vector $e(A)$.

That example is the entire motivation for what follows. The next Part turns it into a theorem and puts a number on it.

Part III

The decode bottleneck, made precise

Part II ended with an example suggesting that writing reasoning out as text throws information away. This Part proves it, bounds it, and — as importantly — says exactly which architectures the bound does and does not apply to. Getting that scoping right matters: the naive version of this argument is wrong, and it is worth seeing why.

10 Three ways a reasoning step can hand off to the next

(a) Separate-call multi-agent (what CHESS, DIN-SQL, DAIL-SQL actually do). Stage k finishes. Its entire internal state is discarded. Stage $k+1$ is a *new call* with a *fresh context* containing stage k's written output as text.

(b) Single-model chain-of-thought. One continuous forward pass. The internal state of every earlier position is kept in the KV cache and remains attendable. But the *input* at each new position is $e(x_t)$: one of $|V|$ fixed vectors.

(c) Latent (ours). One continuous forward pass. The KV cache is kept *and* the input at each new position is an arbitrary vector in $\mathbb{R}^d$, not a row of W_{in}.

The information-theoretic theorem below is about (a) versus (c). The separate argument in Section 14 is about (b) versus (c). Conflating them produces a claim that is easy to disprove, so we keep them apart.

11 Background: entropy and mutual information

Definition 8 (Entropy)**.** For a random variable X taking values x with probabilities $p(x)$,

$$H(X) = -\sum_x p(x) \log_2 p(x)$$

measured in bits. It is the average number of yes/no questions needed to pin X down. A fair coin has $H = 1$; a constant has $H = 0$.

Definition 9 (Mutual information)**.**

$$I(X;Y) = H(X) - H(X \mid Y),$$

the reduction in uncertainty about X from learning Y. It is 0 when they are independent and equals $H(X)$ when Y determines X. It is symmetric and never negative.

Definition 10 (Markov chain)**.** $X \to Y \to Z$ is a Markov chain if Z depends on X only through Y: formally, X and Z are conditionally independent given Y, so $I(X; Z \mid Y) = 0$.

12 The setup, matching what the code and the baselines actually do

Let $H \in \mathbb{R}^d$ be the internal state that stage k has arrived at: everything it figured out. In architecture (a), stage k must convert H into a token sequence

$$T = \text{decode}(H), \qquad |T| = L \text{ tokens},$$

and stage $k+1$ reconstructs its own understanding by reading that text,

$$H' = \text{encode}(T).$$

Stage $k+1$ has no access whatsoever to the original H — the process that produced it has exited and its memory is freed.

> This is not a modelling approximation. It is a literal description of what a separate-call multi-agent pipeline does. That is precisely why the theorem below applies to it.

13 Theorem: text handoff cannot beat its own token count

Theorem 11 (Data Processing Inequality)**.** *If $H \to T \to H'$ is a Markov chain, then*

$$I(H;T) \;\geq\; I(H;H').$$

Proof. Apply the chain rule for mutual information to the three-variable quantity $I(H;\, T, H')$ in two different orders. Splitting off T first:

$$I(H;T,H') = I(H;T) + I(H;H' \mid T).$$

Splitting off H' first:

$$I(H;T,H') = I(H;H') + I(H;T \mid H').$$

Both equal the same quantity, so they equal each other:

$$I(H;T) + I(H;H' \mid T) = I(H;H') + I(H;T \mid H').$$

By the Markov property, $I(H;H' \mid T) = 0$. Hence

$$I(H;T) = I(H;H') + I(H;T \mid H').$$

Every mutual information, including a conditional one, is non-negative (it is an expectation of a Kullback–Leibler divergence, which is ≥ 0). Therefore $I(H;T \mid H') \geq 0$ and $I(H;T) \geq I(H;H')$. □

Lemma 12 (The token channel has a hard ceiling)**.** *If T is a sequence of at most L tokens from a vocabulary V, then*

$$I(H;T) \;\leq\; H(T) \;\leq\; L \log_2 |V| \text{ bits.}$$

Proof. First, $I(H;T) = H(T) - H(T \mid H) \leq H(T)$, because $H(T \mid H) \geq 0$: you cannot share more information with a variable than that variable contains. Second, there are at most $|V|^L$ distinct sequences of length $\leq L$ (there are $|V|$ choices at each of L positions). The entropy of a variable with N possible values is at most $\log_2 N$, attained only when all are equally likely. Hence $H(T) \leq \log_2(|V|^L) = L \log_2 |V|$. □

Corollary 13 (The handoff bound)**.** *In a separate-call multi-agent pipeline, the information the successor stage can possibly recover about its predecessor's internal state is at most*

$$I(H;H') \;\leq\; L \log_2 |V| \text{ bits,}$$

no matter how the text is written, how clever the prompt is, or how large the successor model is.

Proof. Chain Theorem 11 with Lemma 12. □

13.1 Putting our numbers in

Our model has $|V| = 152{,}064$, so

$$\log_2 |V| = \log_2 152064 \approx 17.214 \text{ bits per token.}$$

Example 14 (one role handoff)**.** A rendered teacher role turn in our corpus — a schema-linking block, say — runs to roughly $L = 200$ tokens. The ceiling is

$$200 \times 17.214 \approx 3{,}443 \text{ bits.}$$

Example 15 (the whole transcript)**.** An entire verbalised debate in our traces averages about $L = 900$ generated tokens, giving

$$900 \times 17.214 \approx 15{,}493 \text{ bits}$$

for the complete multi-stage transcript.

Now the other side. In architecture (c) the successor role does not read text. It continues the same forward pass and attends directly to the predecessor's own KV entries. For our model the per-position key/value state is

$$2 \text{ (keys and values)} \times \underbrace{28}_{\text{layers}} \times \underbrace{4}_{\text{KV heads}} \times \underbrace{128}_{\text{head dim}} = 28{,}672 \text{ numbers,}$$

each stored in `bfloat16`, hence at most 16 bits of entropy each:

$$28{,}672 \times 16 = 458{,}752 \text{ bits per position.}$$

The current configuration runs 2 roles $\times$ 2 latent steps = 4 latent positions, so the state the final decode attends to is bounded by

$$4 \times 458{,}752 = 1{,}835{,}008 \approx 1.84 \times 10^6 \text{ bits}$$

of latent working memory — and, critically, *it is not re-encoded.* The successor sees the predecessor's actual tensors, not a reconstruction from prose.

> 1.84×10^6 bits of directly attendable state, versus a 1.5×10^4-bit ceiling on the measured full verbal transcript: a factor of roughly 118 in representational capacity. Neither number measures useful information actually learned.

13.2 What this proves, and — importantly — what it does not

Remark 16 (the comparison is between a bound and an exact value, in the right direction)**.** Corollary 13 is an *upper* bound on the text path. The 1.84×10^6 figure is an upper bound on the four-position latent storage. Comparing two upper bounds proves nothing in general. What rescues the qualitative argument is that the latent handoff is not discretised into generated text: the next step retains access to the preceding KV entries, whereas a separate-call text handoff is capped at $L \log_2 |V|$. The claim "text generation imposes a discrete information ceiling" is sound; the claim "the latent path carries 1.84 million bits of useful reasoning" is not proven and is not claimed.

Remark 17 (the aligned embedding is itself a narrower channel)**.** Section 16 introduces align(h), the map that turns a hidden state back into something usable as an *input* embedding. In its default `soft_embed` form this restricts to the top $k = 32$ logits and returns a convex combination of 32 embedding rows. That is a deterministic function of h, so by the same DPI it can only lose information — it carries roughly the identity of 32 tokens plus 31 free probabilities, on the order of 10^3 bits, not 10^5.

This is not a contradiction: align(h) is the input at the *next* position, and it is a genuinely narrow pipe. The wide channel is the KV cache, which the successor attends to in parallel and which nothing compresses. Both statements are true simultaneously, and stating only the flattering one would be dishonest. `align="identity"` makes the narrow pipe lossless too, and is implemented; it is not the default because a raw hidden state is out of distribution as an input embedding and empirically produced degenerate output.

Remark 18 (capacity is not use)**.** Nothing above shows the model *fills* the latent channel with independent, useful reasoning rather than four near-copies of the same state. That question belongs to training and probing, where the guarantees are far weaker (Part VIII, Part X).

14 The second, different argument: superposition and search

The DPI argument above compares a separate-call pipeline with a latent one. It says nothing about an ordinary, single-model chain-of-thought that retains its KV cache. For that comparison we need a different argument. This section maps the proofs in *Reasoning by Superposition* (Zhu et al., NeurIPS 2025) and *Emergence of Superposition* (Zhu et al., ICLR 2026) onto Text-to-SQL. It then maps the policy-improvement proof in *Latent Thinking Optimization* (Du, Dong and Ning, ICLR 2026) onto execution accuracy.

The conclusion will be deliberately precise:

> Under an explicit finite-search abstraction, continuous thoughts *can* represent every still-viable partial SQL derivation at once and expand that set one step at a time. Thus an idealised transformer can solve the corresponding Text-to-SQL search problem in a number of latent steps equal to the search-graph diameter. This is an expressivity theorem, not proof that our trained checkpoint actually uses this algorithm.

14.1 What the papers actually compare

There are three different questions. They must not be merged.

Question 1: Is continuous CoT more expressive than token-wise CoT? Zhu et al. prove an upper bound for continuous CoT. A two-layer transformer solves directed reachability in D continuous steps. Here D is the graph diameter. Each thought stores the whole reachable frontier.

The best known constant-depth discrete-CoT construction uses $O(n^2)$ decoded steps for an n-vertex graph. This is not a lower bound. The paper says that a smaller discrete construction may exist. Its conclusion explicitly lists a discrete lower bound as future work.

The theoretical comparison is therefore:

$$\begin{array}{ll} \text{continuous CoT} & \text{proved sufficient in } D \text{ steps,} \\ \text{discrete CoT} & \text{best known construction uses } O(n^2) \text{ steps.} \end{array} \tag{3.6a}$$

This is evidence of an expressivity advantage. It is not a strict separation.

Question 2: Is continuous CoT better in the experiment? Yes, on the paper's ProsQA setting. The models are trained from scratch on three-hop and four-hop reachability tasks. The reported results are:

method	transformer depth	test accuracy
continuous CoT	2 layers	near 100%
token-wise CoT	2 layers	about 75%
token-wise CoT*	12 layers	83%
no CoT	2 layers	about 75%

Thus latent superposition is empirically better on this synthetic graph task. The result does not establish the same gap on Text-to-SQL. That transfer is the subject of the conditional reduction below.

Question 3: What does the gradient-descent paper compare? It does not directly compare latent gradient descent with token-wise-CoT gradient descent. It compares two losses inside continuous CoT:

- **COCONUT**: supervise one demonstrated path node.
- **COCONUT-BFS**: supervise the set of reachable frontier nodes.

Their answer accuracy is almost the same on ProsQA. The first paper reports near-perfect accuracy for both. The later paper reports 99.0% for its experimental COCONUT-BFS variant.

Their internal dynamics are not the same. Under single-path COCONUT training, the matching logit μ often converges to a finite value. Under the ideal COCONUT-BFS loss, μ grows at least logarithmically. The exception occurs when the demonstrated path node already has maximum support. Then COCONUT also diverges.

The softmax ratio explains the difference:

$$\frac{q_{\max}}{q_j} = \exp\left(\frac{\mu(d_{\max} - d_j)}{\sqrt{K}}\right). \tag{3.6b}$$

If μ stays finite, several branches retain mass. If $\mu \to \infty$, the non-maximal branches vanish. The distribution collapses onto the maximum-support branches. Thus the single-path latent objective can preserve more branches than explicit BFS supervision. Both objectives can still obtain the same final accuracy on this small benchmark.

Exact conclusion. Continuous CoT is empirically better than token-wise CoT on ProsQA. It also has a better known constructive step bound. A strict theoretical separation is still open. Gradient descent explains why superposition emerges inside continuous CoT. It does not prove a general optimization advantage over token-wise CoT.

14.2 Why Text-to-SQL becomes a search graph

A proof needs a state space. The state cannot be a next token. A token does not say which schema links are still possible. It also does not say which SQL plans remain valid.

The state should contain everything needed for the next decision. This is the Markov requirement. Let h be the full reasoning history. We seek a summary $s(h)$ with this property:

$$p(\text{next refinement} \mid h, x) = p(\text{next refinement} \mid s(h), x). \tag{3.6}$$

This requirement explains the following tuple. It separates observed tool output from model-internal uncertainty instead of pretending that a latent step can query a database.

Definition 19 (Text-to-SQL search state)**.** For a fixed input $x = (q, S, D, e)$, define a state as:

$$s = (v, L, P, Y, A). \tag{3.7}$$

Here v stores values retrieved externally and already present in the prompt. L stores schema-link hypotheses. P stores partial plans, Y stores candidate SQL, and A stores anticipated failure modes learned from offline examples. At inference A is a prediction about likely errors, not an executor response.

The tuple maps onto the external input and the two internal roles in Part V:

component	field	unresolved alternatives
external deterministic retrieval	v	database values written once in the prompt
schema linker	L	columns and join paths
candidate generator	P, Y, A	plans, SQL clauses and anticipated failures

The tuple need not be minimal. It only needs to be sufficient. Two histories with the same tuple expose the same choices to the next stage. Their wording no longer matters. This is why a state abstraction is useful.

Now bound each field. Keep at most k_v values and k_L schema links, and bound plan and SQL length. These bounds make the state set $\mathcal{S}_x = \{s_0, \ldots, s_{N-1}\}$ finite.

A local operation changes one part of the tuple. It may add a schema link, extend a plan, instantiate a predicate, or avoid a failure pattern learned from the training-only mutant outcomes. Write $s_i \to s_j$ when one such operation changes s_i into s_j. This relation forms the edge set $\mathcal{E}_x$. No edge performs a live database call during inference.

Let s_0 contain the unresolved input. Define the goal set as:

$$\mathcal{G}_x = \{s \in \mathcal{S}_x : \mathrm{EX}(Y(s), y^\star, D) = 1\}. \tag{3.8}$$

The pair $G_x = (\mathcal{S}_x, \mathcal{E}_x)$ is the SQL search graph.

The gold query appears in (3.8) only to define correctness for analysis. The model does not receive $y^\star$ at test time. Therefore membership in $\mathcal{G}_x$ cannot be used as a free inference operation. The search proof below shows that the frontier contains every reachable correct state. Selecting one requires a separate verifier.

Example 20 (One branch is one SQL decision)**.** Suppose the question says "visitors who spent more than 50 in total." One state interprets this as `WHERE spent > 50`. Another state interprets it as `GROUP BY visitor` with `HAVING SUM(spent) > 50`. Both states can share the same retrieved values and schema links. They differ in P and Y. The second branch survives the phrase "in total." The first does not.

Proposition 21 (Text-to-SQL is graph reachability)**.** *A correct bounded derivation exists exactly when a goal state is reachable from s_0 in G_x.*

Proof. Take a correct derivation. Record its state after every local operation. Each adjacent pair satisfies $s_i \to s_{i+1}$. The recorded sequence is a path. Its last state belongs to $\mathcal{G}_x$ by (3.8).

Now take a path from s_0 to a goal. Every edge is a permitted local operation. The path is therefore a valid derivation. Its last query has EX = 1. It is correct. This proves both directions. □

14.3 Deriving breadth-first search as vector algebra

Number the states from 0 to $N-1$. Give state s_i the standard basis vector $u_i \in \mathbb{R}^N$. The vector u_i has a 1 in coordinate i and zeros elsewhere. This choice makes membership readable from one coordinate.

Add a self-loop to every state. Define the closed adjacency matrix as:

$$B_{ji} = \begin{cases} 1 & \text{if } i = j \text{ or } s_i \to s_j, \\ 0 & \text{otherwise.} \end{cases} \tag{3.9}$$

The column index is the old state. The row index is the next state. Therefore matrix multiplication sends active states to their successors.

Let $a_c \in \{0,1\}^N$ mark the states reached after at most c steps. Start with $a_0 = u_0$. The exact BFS update is:

$$a_{c+1} = \mathbf{1}[Ba_c > 0]. \tag{3.10}$$

The indicator is applied to each coordinate.

Why does this work? Consider coordinate j before thresholding:

$$(Ba_c)_j = \sum_{i=0}^{N-1} B_{ji}(a_c)_i. \tag{3.11}$$

Every term is zero or one. The sum is positive exactly when an active state s_i equals s_j or points to s_j. The threshold keeps exactly those states. This is one breadth-first expansion.

Lemma 22 (Exact frontier invariant)**.** *Let F_c contain all states reachable in at most c edges. The support of a_c in (3.10) is exactly F_c.*

Proof. At $c = 0$, only s_0 is active. Thus $\operatorname{supp}(a_0) = F_0$.

Assume $\operatorname{supp}(a_c) = F_c$. Equation (3.11) makes coordinate j positive exactly when s_j is already in F_c or has a predecessor in F_c. These are exactly the states in F_{c+1}. Applying the indicator proves $\operatorname{supp}(a_{c+1}) = F_{c+1}$. Induction proves the claim. □

Now let $U = [u_0 \ \cdots \ u_{N-1}]$. Store the frontier in one continuous thought:

$$z_c = Ua_c = \sum_{s_i \in F_c} u_i. \tag{3.12}$$

This formula is not an extra assumption. It is the vector form of the BFS indicator. Orthonormality gives $a_c = U^\top z_c$. Thus no membership bit is lost. Layer normalisation may rescale z_c. It does not change which coordinates are non-zero.

This is why superposition appears. Addition packs many membership indicators into one vector. A discrete token can name one basis vector. A continuous vector can contain many basis directions at once.

14.4 How two transformer layers implement the update

The matrix proof above gives the algorithm. We now realise its operations with attention. This is the missing bridge between BFS and continuous CoT.

Write each graph edge in the prompt as three tokens:

$$\texttt{source}_i \quad \texttt{target}_j \quad \texttt{<edge>}. \tag{3.13}$$

Use three orthogonal subspaces. One stores current-state vectors. One stores edge sources. One stores edge targets.

Layer 1 copies each edge. At an `<edge>` token, one attention head reads two positions back. It copies the source. A second head reads one position back. It copies the target. We now construct those heads.

Give position t a basis vector $p_t \in \mathbb{R}^{T_{\max}}$. For offset ℓ, define a shift map $R_\ell p_t = p_{t-\ell}$. At position t, use the following query and keys:

$$q_t = \gamma R_\ell p_t, \qquad k_j = p_j. \tag{3.14a}$$

Orthonormality gives the score:

$$q_t^\top k_j = \begin{cases} \gamma & \text{if } j = t - \ell, \\ 0 & \text{otherwise.} \end{cases} \tag{3.14b}$$

The softmax weight on the desired position is:

$$\alpha_\star = \frac{e^\gamma}{e^\gamma + T - 1}. \tag{3.14c}$$

For any $0 < \epsilon < 1/2$, choose:

$$\gamma > log \frac{(T-1)(1-\epsilon)}{\epsilon}. \tag{3.14d}$$

Then $\alpha_\star > 1 - \epsilon$. Every wrong position has total weight below ϵ. Choose $\ell = 2$ for the source head. Choose $\ell = 1$ for the target head. Their value maps copy content into separate buffers. A threshold MLP removes the error. It uses the two-ReLU function in (3.17d) with $\delta_0 = \epsilon$ and $\delta_1 = 1 - \epsilon$. The edge marker then holds exactly:

$$h_{ij}^{(1)} = u_i^{\text{src}} + u_j^{\text{tgt}}. \tag{3.14}$$

This proof uses a one-hot positional subspace because it is transparent. Zhu et al. construct the same chooser for sinusoidal positions and RoPE. Their result removes the need for this extra $T_{\max}$-dimensional block.

Layer 2 matches live sources. Let the current thought be:

$$z_c = \sum_{i \in F_c} \lambda_i u_i, \qquad \lambda_i > 0. \tag{3.15}$$

Choose the query map so that u_i becomes u_i^{src}. Choose the key map so that $h_{ij}^{(1)}$ exposes u_i^{src}. Orthonormality then gives the edge score:

$$\left\langle Q z_c, K h_{ij}^{(1)} \right\rangle = \begin{cases} \lambda_i & \text{if } s_i \in F_c, \\ 0 & \text{otherwise.} \end{cases} \tag{3.16}$$

Choose the value map so that $h_{ij}^{(1)}$ returns u_j. Attention now adds the target of every edge whose source is live.

Softmax gives small positive weight to unmatched edges. That does not break the construction. The threshold MLP gives every live coordinate equal weight. After normalisation, $\lambda_c = 1/\sqrt{|F_c|} \geq 1/\sqrt{N}$. There are at most m edge tokens. Scale the scores by γ. Choose one global scale so that:

$$e^{\gamma/\sqrt{N}} > 2m^2. \tag{3.17a}$$

For a true successor, at least one matched edge contributes to its coordinate. Its coefficient is larger than:

$$\delta_1 = \frac{e^{\gamma/\sqrt{N}}}{me^{\gamma/\sqrt{N}} + m} > \frac{1}{2m}. \tag{3.17b}$$

For a false successor, only unmatched edges contribute. Its coefficient is smaller than:

$$\delta_0 = \frac{m}{e^{\gamma/\sqrt{N}}} < \frac{1}{2m}. \tag{3.17c}$$

Thus a fixed gap separates signal from noise. The MLP uses the scalar function:

$$f(x) = \frac{\mathrm{ReLU}(x - \delta_0) - \mathrm{ReLU}(x - \delta_1)}{\delta_1 - \delta_0}. \tag{3.17d}$$

For $x \leq \delta_0$, $f(x) = 0$. For $x \geq \delta_1$, $f(x) = 1$. No coordinate lies inside the gap. Two ReLU units therefore implement the threshold exactly on all states that occur in the construction. Apply f to every state coordinate. The residual connection keeps the old frontier. The attention values add its successors. The MLP converts both to a binary indicator. The output support is F_{c+1}.

We have now constructed the update used in Lemma 22. Layer 1 reads the edge list. Layer 2 performs the matrix multiplication in (3.11). The MLP performs the indicator in (3.10). Nothing called "BFS" was inserted by name. It follows from these three operations.

14.5 The conditional Text-to-SQL theorem

We can now state exactly what transfers from the graph proof and exactly what must be assumed for the transfer.

Assumption 23 (Finite explicit SQL-search encoding)**.** For every input under consideration, G_x is finite. Its encoded edge list fits within a fixed context bound $T_{\max}$. Every search state has a distinct representational basis direction. The required hidden width may scale as $O(|\mathcal{S}_x| + T_{\max})$.

Theorem 24 (Conditional latent breadth-first coverage for Text-to-SQL)**.** *Under Assumption 23, there exist parameters of a two-layer transformer. The parameters are independent of each database instance within the fixed bounds. After C continuous-thought steps, the latent state contains every SQL state reachable within C local refinements. Thus it contains a correct candidate whenever one is reachable within C steps.*

Proof. Assumption 23 supplies the finite edge list and basis directions used in (3.13)–(3.16). The two attention layers and threshold MLP therefore implement (3.10). Lemma 22 shows that z_C has support F_C. Proposition 21 says that any correct candidate at distance at most C belongs to F_C. Therefore its basis direction occurs in z_C. No correctness oracle is used in this coverage claim. □

Corollary 25 (When coverage becomes correct output)**.** *Assume a verifier $V_x(Y)$ is sound and complete on the finite candidate set:*

$$V_x(Y) = 1 \quad \Longleftrightarrow \quad \mathrm{EX}(Y, y^\star, D) = 1. \tag{3.18}$$

If Theorem 24 reaches a correct candidate, applying V_x to the reached complete queries returns a correct SQL query.

Proof. The theorem places the correct candidate in F_C. Completeness makes the verifier accept it. Soundness prevents acceptance of an incorrect candidate. Therefore every accepted output is correct. □

This verifier is a real assumption. Benchmark EX uses the gold result and is not available at deployment. Parser checks and execution errors are available. They are not complete semantic verifiers. Learned unit tests are also imperfect. Thus Theorem 24 proves parallel search coverage. Corollary 25 proves final correctness only when the verifier condition holds.

Remark 26 (where the SQL graph comes from)**.** The graph need not be written out by a human. Many edges follow from the schema. Foreign keys define legal joins. SQL grammar defines legal plan extensions. Externally retrieved values define legal predicates. Offline mutant outcomes can teach the model which apparent refinements tend to cause syntax errors, runtime errors, empty results or result mismatches; they do not create live execution-feedback edges at inference.

The theorem still assumes that these relations are represented inside the model. It proves the search step after that representation exists. It does not prove that our training data makes Qwen recover the full graph. That is a separate learning problem.

Remark 27 (latent slots are not pipeline stages)**.** One graph edge is one local refinement. One role may require several such refinements. Conversely, one transformer step may combine several easy refinements. The two role names therefore do not imply a graph diameter of two. Our four latent positions cover the theorem only when the learned effective diameter is at most four; the theorem does not prove that this bound is sufficient for real Text-to-SQL questions.

For a beginner, the proof says something simple. A textual reasoner commonly writes "use column A" and makes that decision part of the next input. The constructed latent reasoner can instead carry:

$$0.44u_{\text{column A}} + 0.40u_{\text{column B}} + 0.09u_{\text{join path C}} + \cdots$$

and extend *all* non-zero alternatives. Candidate generation chooses the final branch; subsequent GRPO can reward or penalise that decoded query using execution, but the inference-time latent chain itself receives no execution feedback. Early uncertainty is retained rather than prematurely converted into a single symbol.

Corollary 28 (Why the search can be shorter)**.** *Consider a search tree of branching factor b and depth D. Explicit enumeration can require $1 + b + \cdots + b^D = O(b^D)$ hypothesis mentions. The construction in Theorem 24 instead updates one superposition frontier for D recurrent steps.*

Proof. The explicit count is the geometric series. The latent count is one application of (3.10) for each graph depth. Lemma 22 shows that no reachable state is lost. □

This is an advantage in recurrent *steps*, purchased with width: the vector needs enough coordinates to distinguish the relevant states. The source paper proves D continuous steps for graph diameter D, while the best previously known constant-depth discrete-CoT construction uses $O(n^2)$ decoded steps for n vertices. That is an upper-bound separation between constructions, *not* a lower bound proving that every discrete-CoT model needs $\Omega(n^2)$ steps.

14.6 Why ordinary decoded CoT does not get this property for free

Return to a state whose output distribution is $p \approx (0.44, 0.40, 0.09, 0.07)$ over four hypotheses. A decoded step samples or takes an argmax and feeds back one embedding u_A. The continuous step can feed back $\sum_i p_i u_i$. Decoding is therefore a projection from an interior point of the representation space to one member of a finite codebook. Once a branch has zero support in the recurrent state, local expansion of the chosen branch cannot rediscover it unless the model backtracks or independently reconstructs it.

There are two important qualifications. First, textual CoT can write a list of several alternatives. It then pays tokens proportional to that explicit list. Second, an ordinary transformer's KV cache may preserve uncertainty internally despite the decoded token. Consequently, "discrete CoT is always depth-first" would be false. The theorem gives continuous feedback a direct parallel-BFS construction. It does not establish that every textual reasoner is incapable of parallel internal computation.

14.7 Why gradient training can produce the mixture

An existence construction is useful only if learning can approach it. *Emergence of Superposition* analyses a simplified two-layer transformer trained from one demonstrated path, not from labels for the entire BFS frontier. We now derive its main equation.

Let $K = |F_c|$. Use orthonormal target embeddings $E_t(s)$. Use a separate orthonormal source space $E_s(s)$. The current thought is:

$$h_c = \frac{1}{\sqrt{K}} \sum_{s \in F_c} E_t(s). \tag{3.19}$$

An edge $s_i \to s_j$ stores both endpoints:

$$h_{ij} = E_s(s_i) + E_t(s_j). \tag{3.20}$$

The trainable matching matrix uses one scalar μ:

$$W(\mu) = \mu \sum_{s \in \mathcal{S}_x} E_t(s) E_s(s)^\top. \tag{3.21}$$

This form is invented for one reason. It maps a source embedding into the same space as the thought. It maps every wrong source to an orthogonal direction. Therefore the score of edge $s_i \to s_j$ is:

$$h_c^\top W(\mu) h_{ij} = \frac{\mu}{\sqrt{K}} \mathbf{1}[s_i \in F_c]. \tag{3.22}$$

Equation (3.22) is the local search operation. A positive μ rewards edges whose sources are live. A zero μ ignores the graph.

Some next states receive support from several live predecessors. Define their support count as:

$$d_j = \sum_{s_i \in F_c} \mathbf{1}[s_i \to s_j]. \tag{3.23}$$

After aggregating edge values, the part of the logit controlled by μ is:

$$\xi_j(\mu) = b_j + \frac{\mu d_j}{\sqrt{K}}. \tag{3.24}$$

The term b_j contains residual information. It does not depend on μ.

Training presents one demonstrated next state $s^\star$. Let $d^\star = d_{s^\star}$. Its softmax probability and loss are:

$$q_j(\mu) = \frac{e^{\xi_j(\mu)}}{\sum_k e^{\xi_k(\mu)}}, \qquad \ell(\mu) = -\log q_{s^\star}(\mu). \tag{3.25}$$

Differentiate the loss. The derivative of log-sum-exp is the softmax-weighted mean. Therefore:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sqrt{K}} \left(\sum_j q_j(\mu) d_j - d^\star \right). \tag{3.26}$$

Define the expected support count as:

$$F(\mu) = \sum_j q_j(\mu) d_j. \tag{3.27}$$

Under gradient flow, $\dot{\mu} = -\alpha \, \partial \ell / \partial \mu$. Dataset averaging contributes the paper's factor $1/n$. Thus:

$$\dot{\mu}(t) = \frac{\alpha}{n\sqrt{K}} \big(d^\star - F(\mu(t)) \big). \tag{3.28}$$

This is where the differential equation comes from. It is not an independent assumption.

We also need to know how F moves. Differentiate (3.27):

$$F'(\mu) = \frac{1}{\sqrt{K}} \left(\sum_j q_j d_j^2 - \left(\sum_j q_j d_j \right)^2 \right) = \frac{1}{\sqrt{K}} \operatorname{Var}_{q(\mu)}(d_j) \geq 0. \tag{3.29}$$

The variance is positive unless every candidate has the same support count. Hence F is increasing in the non-degenerate case. As $\mu \to \infty$, softmax concentrates on the largest d_j. Therefore $F(\mu) \to d_{\max}$.

Assume the demonstrated state has an intermediate support count:

$$F(0) < d^\star < d_{\max}. \tag{3.30}$$

Continuity gives a finite $\mu^\star$ with $F(\mu^\star) = d^\star$. Strict increase makes it unique.

Proposition 29 (Conditional emergence for SQL states)**.** *Under the model used to derive (3.28), condition (3.30) implies $\mu(t) \to \mu^\star < \infty$. The next continuous thought assigns strictly positive weight to every locally reachable SQL state rather than becoming one-hot on the most locally supported state.*

Proof. Because F is strictly increasing, the right side of (3.28) is positive below $\mu^\star$, zero at $\mu^\star$, and negative above it. Gradient flow therefore moves toward that stable fixed point.

Linear attention gives the following weight for each next state s_j:

$$\beta_j = \lambda_j \mathbf{1}[s_j \in F_c] + \mu^\star \sum_{s_i \in F_c} \lambda_i \mathbf{1}[s_i \to s_j]. \tag{3.31}$$

All live weights and $\mu^\star$ are positive. Every genuine successor therefore has $\beta_j > 0$. A finite $\mu^\star$ also keeps softmax ratios finite. Lower support branches retain non-zero mass. This proves the claim. □

Equation (3.31) explains the Text-to-SQL effect. The model exploits local structure because $\mu^\star > 0$. It can keep several joins or plans because $\mu^\star$ is finite. The proof is conditional. Our model uses many layers, softmax attention, non-orthogonal learned representations, LoRA, final-SQL supervision, and only four latent positions. Therefore the theorem explains a plausible learned mechanism. It does not certify that the mechanism emerged in our checkpoint.

14.8 A separate guarantee: reward latent SQL trajectories by execution

The third paper proves something different. It does not prove breadth-first search. It proves that if we can sample alternative latent trajectories and score them, KL-regularised reward reweighting increases the probability of good ones. Here the translation to Text-to-SQL is exact because we already possess an executable correctness signal.

Let z be a complete latent trajectory for input x, let $Y(z)$ be the SQL it decodes, and define the ideal reward as:

$$r^\star(x, z) = \mathrm{EX}(Y(z), y^\star, D) \in \{0, 1\}. \tag{3.32}$$

Maximising reward alone places all mass on the highest-scoring sample. This is unsafe when the reward model is imperfect. The KL term limits movement away from the base policy. For a reference policy $\pi_{\mathrm{ref}}(z \mid x)$, optimise:

$$\max_{\pi} \quad \mathbb{E}_{z\sim\pi}[r(x, z)] - \beta D_{\mathrm{KL}}(\pi \| \pi_{\mathrm{ref}}), \qquad \beta > 0. \tag{3.33}$$

A small β trusts the reward. A large β stays near the base model.

Theorem 30 (Execution-reward tilt)**.** *Consider a finite sampled set of latent SQL trajectories. Assume $\pi_{\mathrm{ref}}(z \mid x) > 0$ on this set. The unique solution of (3.33) is:*

$$\pi_r(z \mid x) = \frac{\pi_{\mathrm{ref}}(z \mid x) e^{r(x,z)/\beta}}{\sum_{z'} \pi_{\mathrm{ref}}(z' \mid x) e^{r(x,z')/\beta}}. \tag{3.34}$$

If $r = r^\star$ and the reference probability of an execution-correct trajectory is $p \in (0, 1)$, its probability after reweighting is:

$$p_\beta = \frac{p e^{1/\beta}}{p e^{1/\beta} + 1 - p} > p. \tag{3.35}$$

Thus ideal latent-trajectory selection strictly improves expected execution accuracy whenever both correct and incorrect trajectories have non-zero mass.

Proof. Write $\pi_z = \pi(z \mid x)$ and $\pi_z^0 = \pi_{\mathrm{ref}}(z \mid x)$. Add a multiplier η for $\sum_z \pi_z = 1$. The Lagrangian is:

$$\mathcal{J}(\pi, \eta) = \sum_z \pi_z r_z - \beta \sum_z \pi_z \log \frac{\pi_z}{\pi_z^0} + \eta \left(\sum_z \pi_z - 1 \right). \tag{3.36}$$

Differentiate with respect to π_z. Set the result to zero:

$$r_z - \beta \left(\log \frac{\pi_z}{\pi_z^0} + 1 \right) + \eta = 0. \tag{3.37}$$

Rearrange (3.37). Terms independent of z become one normalising constant. This gives $\pi_z \propto \pi_z^0 e^{r_z/\beta}$. Normalisation gives (3.34). The Hessian has diagonal entries $-\beta/\pi_z < 0$. The objective is strictly concave on the probability simplex. Therefore this stationary point is the unique maximum.

Now use binary execution reward. Every correct trajectory gains the factor $e^{1/\beta}$. Every incorrect trajectory gains the factor 1. Their base masses are p and $1-p$. Normalisation gives (3.35). Finally:

$$p_\beta - p = \frac{p(1-p)(e^{1/\beta} - 1)}{p e^{1/\beta} + 1 - p} > 0. \tag{3.38}$$

□

Now suppose $|r - r^\star| \le \epsilon$. Let Z_r and $Z_\star$ be the two normalising constants in (3.34). The reward bound gives:

$$e^{-\epsilon/\beta} Z_\star \le Z_r \le e^{\epsilon/\beta} Z_\star. \tag{3.39}$$

Hence the log density ratio obeys:

$$\left| \log \frac{\pi_r(z \mid x)}{\pi_{r^\star}(z \mid x)} \right| \le \frac{2\epsilon}{\beta}. \tag{3.40}$$

Averaging (3.40) under π_r gives $D_{\mathrm{KL}}(\pi_r \| \pi_{r^\star}) \le 2\epsilon/\beta$. Pinsker's inequality then gives:

$$\left| \mathbb{E}_{\pi_r} r^\star - \mathbb{E}_{\pi_{r^\star}} r^\star \right| \le \sqrt{\frac{4\epsilon}{\beta}}. \tag{3.41}$$

This is the bound in Du et al. A small β sharpens selection. It also amplifies reward error. Execution provides strong labels in Text-to-SQL. Yet one database is not a perfect semantic oracle. Section X explains the spurious-correctness problem.

This theorem requires variation in z. Sampling several SQL strings from one deterministic latent prefix is not enough. Every member then has the same z. A latent reward cannot choose among latent reasoning paths. The present GRPO code has that shared-prefix structure. See Proposition 76. Applying Theorem 30 literally would require stochastic latent transitions, dropout masks, or separately perturbed latent rollouts before SQL decoding.

14.9 What the three proofs jointly establish—and what they do not

The chain of logic is now complete:

$$\boxed{\text{Text-to-SQL derivation}} \xrightarrow{\text{Proposition 21}} \boxed{\text{reachability}}$$
$$\xrightarrow{\text{Theorem 24}} \boxed{\text{parallel latent frontier}} \xrightarrow{\text{Theorem 30}} \boxed{\text{execution-guided selection}}.$$

The first arrow is an exact reduction after a finite search bound. The second is a conditional expressivity theorem inherited from the graph construction. The training-dynamics result explains how a superposition can emerge from a single demonstrated path under its simplified assumptions. The final arrow is an exact policy-improvement result if distinct latent trajectories are actually sampled and their reward is accurate.

The papers do not prove universal dominance over discrete CoT. They do not prove that this checkpoint implements BFS. They also do not prove that four latent slots cover every SQL search graph. Their result is narrower. Latent recurrence can postpone early schema and program choices and can represent several plausible derivations in parallel. Correctness signals can then improve the decoded selection. The current two-role experiment must establish empirically whether this architecture helps; legacy six-role measurements cannot answer that question. A frontier-support probe is still needed to identify the mechanism.

Part IV
The continuous thought

We now build the mechanism itself, in four steps, each using only the previous one.

15 Step 1: latent mode

Definition 31 (Language mode and latent mode)**.** A model in **language mode** at position t does

$$h_t \;\to\; \text{softmax}(W_{\text{out}} h_t) \;\to\; x_{t+1} \;\to\; e(x_{t+1}),$$

i.e. equation (2.3). A model in **latent mode** skips the middle entirely:

$$\boxed{\text{next input embedding} \;=\; \text{align}(h_t)} \tag{4.1}$$

No token is sampled, no token is emitted, and the position contributes nothing to the output text.

Coconut states the same thing as a modification of the input matrix. If latent mode runs between positions i and j, then for $i < t < j$,

$$E_t = \big[e(x_1), \ldots, e(x_i),\; h_i,\; h_{i+1}, \ldots, h_{t-1}\big],$$

and once latent mode ends the input reverts to ordinary embeddings:

$$E_t = \big[e(x_1), \ldots, e(x_i),\; h_i, \ldots, h_{j-1},\; e(x_j), \ldots, e(x_t)\big].$$

The vector h_t produced in latent mode is called a **continuous thought**. The quantity $p(x_{t+1} \mid x_{\le t})$ is simply not defined at those positions: nothing is being predicted there. (One *can* still compute $\text{softmax}(W_{\text{out}} h_t)$ for diagnostic purposes, which is how Coconut probes what a latent state is "thinking"; nothing in training uses it.)

> A latent step costs **one forward pass** and emits **zero tokens**. That is the entire efficiency claim, and it is exact, not statistical.

16 Step 2: the alignment map

Equation (4.1) has an `align` in it. Why not feed h_t back directly?

Because h_t and $e(x)$ live in the same space $\mathbb{R}^d$ but are not the same *kind* of object. $e(x)$ is a row of W_{in}: it comes from a set of 152,064 specific vectors that the first transformer layer has seen during all of pretraining. h_t is an output of the last layer after final normalisation. Feeding one where the other is expected is an *out-of-distribution* input. The project measured this: with the identity map the model produced degenerate token soup at evaluation.

Three constructions are implemented in `latentsql/distill/latent.py::build_aligner`.

16.1 Construction 1: identity (Coconut's own choice)

$$\text{align}(h) = h.$$

Kept for ablation. It is the only construction that is information-lossless in the narrow-pipe sense of Section 13.2.

16.2 Construction 2: linear

The natural way to get from hidden space back to input space is to go through the vocabulary and come back:

$$h \xrightarrow{W_{\text{out}}} \text{logits} \in \mathbb{R}^{|V|} \xrightarrow{W_{\text{in}}} \mathbb{R}^d.$$

Composing the two matrices once, up front, gives a single $d \times d$ matrix:

$$W_a = W_{\text{out}}^\top W_{\text{in}} \in \mathbb{R}^{d \times d}, \qquad \text{align}(h) = h\, W_a. \tag{4.2}$$

Remark 32 (a scale problem, and the fix)**.** W_a sums over all 152,064 vocabulary entries, so hW_a has a norm orders of magnitude away from that of a real token embedding — feeding it in directly would present the model with either a near-zero or an exploding input. The code therefore rescales once at construction time:

$$W_a \;\leftarrow\; W_a \cdot \frac{\overline{\|W_{\text{in}}[v,:]\|}}{\overline{\|zW_a\|}}, \qquad z \sim \mathcal{N}(0, I),$$

i.e. it measures the average norm of a real input embedding, measures the average norm W_a produces on random probes, and scales so the two agree.

Remark 33 (relation to the published form)**.** LatentMAS writes the alignment as $W_a \approx W_{\text{out}}^{+} W_{\text{in}}$ with a pseudo-inverse. With the shape convention logits $= hW_{\text{out}}^\top$ used here, the composition that actually maps hidden $\to$ vocabulary $\to$ embedding is $W_{\text{out}}^\top W_{\text{in}}$, which is what is implemented. The two agree up to a logit temperature.

16.3 Construction 3: soft embedding (the default)

Rather than a first-order linear surrogate, take the token distribution the hidden state actually encodes, restricted to its top k entries, and return the corresponding convex combination of embedding rows:

$$\ell = W_{\text{out}} h, \qquad \mathcal{K} = \text{top-}k(\ell), \qquad \pi_v = \frac{\exp(\ell_v/\tau)}{\sum_{u \in \mathcal{K}} \exp(\ell_u/\tau)} \;\; (v \in \mathcal{K}),$$

$$\boxed{\text{align}_{\text{soft}}(h) \;=\; \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v,:]} \tag{4.3}$$

with $k = 32$ and $\tau = 1$ by default.

> Equation (4.3) is precisely the "keep the frontier" object of Section 14, written down. It is a weighted blend of the 32 tokens the model currently considers plausible — a point in embedding space that no single token occupies — and it is guaranteed in distribution, because it is a convex combination of real rows of W_{in}.

Example 34 (the toy model again)**.** With $\ell = (2.0, 1.9, 0.1, 0.0)$ over $\{A, B, C, D\}$ and $k = 2$:

$$\pi_A = \frac{e^{2.0}}{e^{2.0} + e^{1.9}} = \frac{7.389}{7.389 + 6.686} = 0.5250, \qquad \pi_B = 0.4750,$$

so

$$\text{align}_{\text{soft}}(h) = 0.5250\, e(A) + 0.4750\, e(B).$$

Greedy decoding would have produced $e(A)$ and discarded B entirely.

A definition needed before the lemma: convex combination and convex hull

The lemma below uses two terms that are not assumed known. Both are simple, and both are worth building from a picture rather than a formula first.

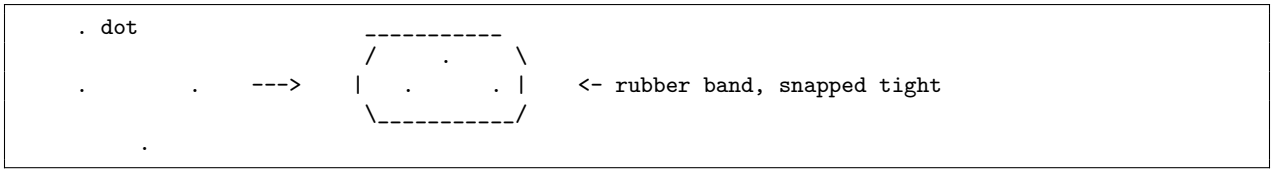
Definition 35 (Convex combination)**.** A **convex combination** of points $p_1, \ldots, p_K$ is any weighted average

$$w_1 p_1 + w_2 p_2 + \cdots + w_K p_K, \qquad w_i \geq 0, \qquad \sum_{i=1}^{K} w_i = 1.$$

The two conditions on the weights matter individually. Take two points A and B on a page. If the weights must be non-negative and sum to 1, the only points you can reach by mixing A and B are the ones *on the straight line segment between them* — $0.5A + 0.5B$ is the midpoint, $0.9A + 0.1B$ is a point close to A, and so on. Drop the "non-negative" requirement and you could take a weight of -0.5 on B, which points *away* from B and shoots straight past A out the far side — off the segment entirely. Drop the "sum to 1" requirement and $2A + 2B$ scales the whole thing outward, again leaving the segment. Both restrictions together are exactly what keeps a weighted mix trapped between the points, never past them.

Definition 36 (Convex hull)**.** The **convex hull** of a set of points is every point reachable as a convex combination of some (or all) of them.

Picture a handful of dots scattered on a page, and a rubber band stretched around the outermost ones and let snap tight, as drawn below. Every point trapped inside the band — the dots themselves and everything between them — is the convex hull. A dot already surrounded by others adds nothing new; the band only bends around the outermost ones. And this is not just an analogy: it is exactly what Definition 35 says algebraically — non-negative weights summing to 1 is precisely what keeps a mix from escaping the band.



Now scale the picture up. Instead of a handful of dots on a 2-number page, our vocabulary gives 152,064 "dots" (one embedding row per token), each described by 3584 numbers instead of 2. The idea does not change — there is still an (imaginary, 3584-dimensional) rubber band stretched around all 152,064 of them, and the region it traps is the convex hull of $\{W_{\text{in}}[v,:] : v \in V\}$: precisely *the neighbourhood every real token embedding lives in*, plus every point reachable by non-negative-weighted mixes of them.

Setup, so the lemmas below are about h and W_{in} explicitly. Fix a hidden state $h \in \mathbb{R}^d$. It determines a logit vector $\ell = W_{\text{out}} h \in \mathbb{R}^{|V|}$, a top-$k$ index set $\mathcal{K} = \mathcal{K}(h) = \text{top-}k(\ell)$, and hence a *finite, h-dependent* candidate set of embedding rows

$$S_h \;:=\; \{\, W_{\text{in}}[v,:] \;:\; v \in \mathcal{K}(h) \,\} \subset \mathbb{R}^d, \qquad |S_h| = k = 32.$$

By (4.3), $\text{align}_{\text{soft}}(h) = \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v,:]$ is a convex combination of exactly the 32 points of S_h — *not* of all 152,064 rows of W_{in}. That distinction is what the weak lemma below misses and the two lemmas after it repair.

Lemma 37 (weak containment)**.** $\mathrm{align}_{\mathrm{soft}}(h) \in \mathrm{conv}\big(\{W_{\mathrm{in}}[v,:] : v \in V\}\big)$.

Proof. The π_v are non-negative and sum to 1 (softmax), so (4.3) is a convex combination of the points of S_h; and $S_h \subseteq \{W_{\mathrm{in}}[v,:] : v \in V\}$, so $\mathrm{conv}(S_h) \subseteq \mathrm{conv}\big(\{W_{\mathrm{in}}[v,:] : v \in V\}\big)$ by monotonicity of the convex-hull operator under set inclusion. □

This says only "inside the rubber band around all 152,064 tokens". With $|V| = 152{,}064 \gg d = 3584$, that hull is generically full-dimensional — it fills essentially the entire 3584-number space — so Lemma 37 alone rules out almost nothing. The next two lemmas use the actual value of h, not just its existence, to say something much smaller.

Lemma 38 (sharp containment: a 31-simplex, not the whole hull)**.** $\mathrm{align}_{\mathrm{soft}}(h) \in \mathrm{conv}(S_h)$*, and generically (when the* 32 *points of* S_h *are affinely independent, the typical case since* $d = 3584 \gg k - 1 = 31$*) this set is a* 31*-dimensional simplex — the convex hull of exactly* 32 *vertices, an object of dimension* 31 *sitting inside a* 3584*-dimensional ambient space.*

Proof. Let $E_h \in \mathbb{R}^{d\times k}$ be the matrix whose i-th column is the i-th row of S_h, and let $\Delta^{k-1} = \{w \in \mathbb{R}^k : w_i \geq 0, \sum_i w_i = 1\}$ be the standard simplex. The map $\phi_h : \Delta^{k-1} \to \mathbb{R}^d$, $\phi_h(w) = E_h w$, is linear and, by definition of a convex hull, $\phi_h(\Delta^{k-1}) = \mathrm{conv}(S_h)$. The softmax weight vector $\pi = (\pi_v)_{v\in\mathcal{K}}$ satisfies $\pi \in \Delta^{k-1}$ (non-negative, sums to 1), and $\mathrm{align}_{\mathrm{soft}}(h) = \phi_h(\pi)$ by (4.3). So $\mathrm{align}_{\mathrm{soft}}(h) \in \phi_h(\Delta^{k-1}) = \mathrm{conv}(S_h)$. Δ^{k-1} has affine dimension $k - 1 = 31$; an injective linear map preserves affine dimension, and 32 points in general position in $\mathbb{R}^{3584}$ are affinely independent (a codimension-0, i.e. generic, condition), so ϕ_h is injective and $\mathrm{conv}(S_h)$ has dimension 31. □

Lemma 39 (genuine mixing, not a corner)**.** *For any finite temperature* $\tau > 0$*, every* $\pi_v > 0$ *strictly, so* $\mathrm{align}_{\mathrm{soft}}(h)$ *lies in the* relative interior *of* $\mathrm{conv}(S_h)$ *— never on a face, edge, or vertex of the simplex.*

Proof. $\pi_v = \exp(\ell_v/\tau)/\sum_{u\in\mathcal{K}} \exp(\ell_u/\tau)$, and $\exp(x) > 0$ for every finite real x, so every $\pi_v > 0$. A convex combination with a strictly positive coefficient on *every* vertex of a simplex is, by definition, in that simplex's relative interior (a point on a proper face has coefficient exactly 0 on at least one vertex). □

Lemma 40 (quantitative localisation: how far from the model's own top guess)**.** *Let* $v^\star = \arg\max_{v\in\mathcal{K}} \pi_v$ *be the single most likely token. Then*

$$\big\|\, \mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:] \,\big\| \;\leq\; \max_{v\in\mathcal{K}} \big\|\, W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:] \,\big\|.$$

Proof. Since $\sum_{v\in\mathcal{K}} \pi_v = 1$, subtract $W_{\mathrm{in}}[v^\star,:]$ from (4.3) using $\pi_v W_{\mathrm{in}}[v^\star,:]$ summed over v equal to $W_{\mathrm{in}}[v^\star,:]$:

$$\mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:] = \sum_{v\in\mathcal{K}} \pi_v \big(W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big).$$

Apply the triangle inequality, then $\pi_v \geq 0$ and $\sum_v \pi_v = 1$:

$$\big\|\mathrm{align}_{\mathrm{soft}}(h) - W_{\mathrm{in}}[v^\star,:]\big\| \leq \sum_{v\in\mathcal{K}} \pi_v \big\|W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big\| \leq \Big(\max_{v\in\mathcal{K}} \big\|W_{\mathrm{in}}[v,:] - W_{\mathrm{in}}[v^\star,:]\big\|\Big) \sum_{v\in\mathcal{K}} \pi_v,$$

and the last sum is 1. □

What the four lemmas jointly say. $\text{align}_{\text{soft}}(h)$ is not merely "somewhere in the 152,064-token hull" (Lemma 37, weak). It is a specific, strictly-interior point (Lemma 39) of a 31-dimensional simplex spanned by only the 32 embeddings h *itself* finds most plausible (Lemma 38), at a distance from the model's own greedy top-1 choice that never exceeds the local spread of those same 32 candidates (Lemma 40). `identity` mode (Construction 1) has none of these guarantees: h is the *last* layer's output, unrelated by construction to the 152,064 rows of W_{in} the *first* layer was trained on, so no lemma above applies to it at all — consistent with the degenerate token soup measured at evaluation.

17 Step 3: latent working memory (the KV cache)

A transformer is causal, so when position $t+1$ is computed, the keys and values of positions $1..t$ are unchanged. They are cached rather than recomputed. That cache is

$$\mathcal{C}_t = \{(K_{1:t}^{(\ell)}, V_{1:t}^{(\ell)})\}_{\ell=1}^{28},$$

one key/value pair per layer per position.

In our chain we carry $\mathcal{C}$ across role boundaries. That is exactly the statement that role $k+1$ attends to role k's actual representations rather than to a re-encoding of role k's prose. It is the wide channel of Section 13.2.

`share_working_memory:`	`true` keeps $\mathcal{C}$ across roles.
`share_working_memory:`	`false` resets it at each role boundary — the ablation that simulates a separate-call pipeline.

17.1 A subtlety the code had to solve: cache mutation under checkpointing

This subsection can be skipped on a first reading, but it is the kind of detail that determines whether the gradients are correct.

To fit training in memory, the m latent steps of a role are wrapped in `torch.utils.checkpoint`: their intermediate activations are not stored, they are *recomputed* during the backward pass. Non-reentrant checkpointing therefore calls the wrapped function **twice** — once now, once during backward.

The problem: the KV cache is mutated *in place*. Each `model(..., use_cache=True)` appends to it. A blind second call would append the same m steps again, so the cache length would be wrong for every role that already ran on top of it.

The fix, at `latent.py:220`, is one line at the top of the checkpointed function:

```
past_key_values.crop(start_len)   # no-op first call; undoes 2nd call's own append
```

`crop` is a plain tensor slice, hence differentiable, and a true no-op when the cache is already at that length. So:

- **First (real) call:** the cache is already at `start_len`; cropping does nothing; m steps are appended.
- **Second (backward-time) call:** forward has completed, so the cache is at `start_len`+m; cropping removes exactly this role's own extension, and the m steps are then redone identically.

The property being restored is **idempotence**: running the block once and running it twice must leave the same state. That is what non-reentrant checkpointing requires, and it is verified in `tests/test_core.py` by comparing LoRA gradients computed with and without checkpointing — they agree to float32 tolerance.

18 Step 4: the cost accounting

Let R be the number of latent roles and m the number of latent steps per role. Then per question:

	Explicit pipeline	**Latent (ours)**
Model calls	one per stage ($\approx$ 6–12)	one
Forward passes	one per generated token, per stage	Rm + one per SQL token
Tokens generated	≈ 1339 (measured)	≈ 61 (measured)
Tokens read	each stage re-reads the schema	schema read once
Cross-stage channel	text, $\leq L \log_2 \lvert V \rvert$ bits	KV cache, no re-encoding

With $R = 6$ and $m = 2$ this is 12 latent forward passes — fewer than the cost of generating 12 tokens, and it replaces roughly 1280 of them.

Remark 41 (what the token counts in this document mean)**.** `Avg tokens` is prompt_tokens + completion_tokens. Latent steps are *not* counted as completion tokens, because they are not tokens; they are tracked separately as `extra.latent_steps`. Anywhere a comparison could be read as hiding the latent cost, both numbers are given.

19 Is this beam search? Is it tree search? A precise comparison

A latent step (4.3) looks at the top-k next-token candidates and reasons about several of them at once. That invites a natural question: is this secretly beam search, or Monte Carlo Tree Search (MCTS), under a different name? **No, to both** — and the reason is worth stating precisely, since "explore several options" and "search over several options" sound alike but are mechanically opposite.

19.1 Before any formal definition: the exact same numbers, two ways

Reuse the worked example from Construction 3 (§16): a hidden state h whose top-2 candidates are tokens A and B, with softmax weights $\pi_A = 0.525$, $\pi_B = 0.475$ (worked out there in full). Trace one step forward, the beam-search way and our way, side by side.

Beam search, $K = 2$, starting from this exact point		
Step	What happens	Result
1	Keep the top 2 tokens as **two separate, complete copies** of the sequence so far — one ending in A, one ending in B.	Two sequences now exist, each with its own KV cache.
2	Run the model **twice**: once continuing from "$\ldots A$", once from "$\ldots B$". Say this gives $(0.6, 0.4)$ over $\{X, Y\}$ after A, and $(0.3, 0.7)$ over $\{X, Y\}$ after B.	Four candidate two-token sequences exist: AX, AY, BX, BY, each with its own cumulative score.
3	Score all four, keep only the best 2 — say BY and AX win.	AY and BX are **thrown away for good**. If the best possible completion started with BX, it is gone, even though B itself was kept.

Our latent step, at the exact same point		
Step	What happens	Result
1	Do not pick A or B. Blend them into one vector: $\text{align}_{\text{soft}}(h) = 0.525\, e(A) + 0.475\, e(B)$.	**One** sequence still exists, one KV cache.
2	Run the model **once**, on that one blended vector.	One next-state distribution comes out. There was never a second, separate "B-branch" sitting around to compare against or discard.
3	(there is no step 3 — nothing was kept apart to choose between)	Nothing is ever thrown away, because nothing was ever split off in the first place.

The whole difference, in one line. Beam search keeps A's branch and B's branch alive as two separate things, extends both, and only *later* throws one away. The latent step never lets A and B become separate things to begin with — so there is nothing later to compare, nothing to discard, and no second forward pass was ever spent computing the branch that beam search would have thrown away anyway.

This is exactly what the formal claim below (Proposition 44) says: at no point in the second table does a second copy of anything exist.

Definition 42 (Beam search)**.** Maintain K separate, explicit token sequences ("beams"). At every step, extend each of the K beams by its own candidate next tokens, score every resulting continuation by cumulative log-probability, and keep only the top K continuations for the next step. At the end, the single highest-scoring completed sequence is returned.

Definition 43 (Monte Carlo Tree Search)**.** Build an explicit tree whose nodes are partial sequences. Repeatedly: descend the tree toward promising nodes (*selection*), extend a leaf with a new candidate (*expansion*), estimate that candidate's value by rolling forward to a full sequence and scoring it (*simulation*), and propagate that score back up the tree to update every ancestor's estimate (*backpropagation*). After many such iterations, commit to the highest-value child at the root.

Proposition 44 (the latent step is neither)**.** *At every position in the sequence (4.3), (5.1), there is exactly* ***one*** *input embedding, one hidden state, one KV cache. No step in* `_latent_steps` *or* `align_soft` *creates a second copy of the sequence, scores two candidates against each other, or discards one in favour of another.*

Proof. Immediate from the definition of $\text{align}_{\text{soft}}$ (4.3): it is a single function $\mathbb{R}^d \to \mathbb{R}^d$, returning one vector $\sum_{v \in \mathcal{K}} \pi_v W_{\text{in}}[v, :]$. There is no data structure in `_latent_steps` (Part IV) holding more than one hypothesis: `pieces` is a single growing list, `past_key_values` a single cache. Beam search and MCTS both require, by their own definitions above, more than one candidate to exist *simultaneously* so that they can be compared; here only one ever exists. □

	Beam search / MCTS	**GRPO** (Part X)	**Latent step (ours)**
When it runs	inference time	training time	inference *and* training
How many sequences exist at once	K, kept explicit and compared	G full rollouts, compared *after* decoding	1 — always
What changes as a result	which sequence is *output*	the model's *weights*	nothing is compared; the state itself is a blend
Extra inference cost	$K\times$ or more forward passes	none (training-only)	**zero** — same cost as one token

The one-sentence answer. Beam search and MCTS get their exploration by keeping K things separate and comparing them; the latent step gets a *different* kind of exploration — Section 14's superposition — by keeping exactly 1 thing and making that one thing a blend. It is cheap for exactly the reason it cannot literally be a search: there is nothing to compare, because nothing was ever kept apart.

Remark 45 (why not use real search here)**.** MCTS-style search at inference would reintroduce precisely the cost this project exists to remove: CHESS's own multi-candidate generation (Part I) already pays several LLM calls per question for exactly this reason. A value estimate for a *partial* SQL query or a partial latent state is also not cheaply available the way it is in a board game (no partial query can be executed against the database to check if it is "winning"), which is the other standard prerequisite MCTS needs and this setting does not supply for free.

Remark 46 (and the final SQL decode is not a search either)**.** The one place a real, discrete choice *is* made is decoding the SQL tokens after `<|consensus|>` (Part V). Evaluation runs this at `temperature=0` (`configs/latent_sft_1000_7b_bf16.yaml`), so `LatentReasoner.generate` takes a plain greedy argmax at every position (`latent.py`, the decode loop after the consensus tag) — one path, no beam, no rescoring. GRPO's `group_size` rollouts (Part X) do sample G different SQL completions per prompt, but only to *compute a training reward*; by Proposition 76, every one of those G completions is decoded from the identical shared latent prefix, so even there no search over *latent* states occurs — only the decoded text differs.

Part V
From a multi-agent pipeline to a chain of latent slots

Part IV built one latent step. This Part says what the latent steps are *for*.

20 What remains outside the student

Value retrieval is a tool operation. The system deterministically extracts keywords from the question, looks them up in the database value index, and inserts the returned value hints once in the student prompt. The same retrieval function and cache are used when preparing SFT records, constructing GRPO tasks, evaluating checkpoints and probing latent states. Retrieval is therefore neither a latent role nor a counted reasoning step, and the student is not expected to infer values it cannot read from the database.

Execution is also external. The student receives no parser output, result table or database error while producing a test query. Execution is used before SFT to label offline examples, during GRPO to compute rewards, and after generation to compute EX and true EX. This distinction licenses the claim that training teaches the model to *anticipate execution failures*; it does not license the claim that the deployed latent chain *uses execution feedback*.

21 How the SFT records are selected

The teacher may perform a richer CHESS process, but the latent corpus keeps only round-0 first-pass CHESS candidates whose ordinary execution score is EX = 1. Later revisions are excluded: otherwise a "candidate-generation" target could quietly contain information obtained only after executing an earlier query. Each retained record ends with the correct first-pass candidate and the textual training label `Outcome: valid --- EX=1`.

The candidate turn also contains up to two deterministic one-edit mutants of the correct SQL. Each mutant is executed offline and labelled as `syntax_error`, `runtime_error`, `empty_result`, or `result_mismatch`. These labels are training-only supervision embedded in the candidate-generation trace; they are not extra roles, are not produced by an LLM, and are never supplied by an executor at inference. The positive endpoint remains the correct first-pass EX = 1 SQL.

22 The two-role chain

The current internal chain contains exactly two roles plus a terminator:

$$\rho = \big(\underbrace{r_1, r_2}_{\text{latent-trainable}} , \mathsf{consensus} \big), \qquad r_1 = \texttt{schema_linker}, \quad r_2 = \texttt{candidate_generator}.$$

k	role r_k	training responsibility
1	`schema_linker`	identify relevant tables, columns and join paths using the schema and externally retrieved values
2	`candidate_generator`	form the SQL, contrast it with labelled one-edit failure examples, and end at the correct first-pass candidate
–	`consensus`	*not* a reasoning stage; the marker after which SQL is emitted

There is deliberately no information-retriever latent role, because retrieval requires the value index; no DIN-SQL query-classifier role, because its label is a routing switch for a multi-prompt pipeline and the discarded decomposition supplied too little supervision; and no query-fixer or unit-tester latent role, because those stages would require live execution feedback. Revision and verification pressure enters later through GRPO rewards applied to the final decoded query.

> `consensus` is punctuation, not a third agent. It says "the latent computation is over; decode the query now."

23 The tag vocabulary and sequence

Each role is announced by a literal string, $\mathsf{tag}(r) =$ `<|agent r|>`, and termination is announced by $\mathsf{tag}(\mathsf{consensus}) =$ `<|consensus|>`. These tags are not decoration: they anchor which of the two internal operations the following latent states should perform.

Let c be the rendered prompt containing the schema, question, benchmark evidence and one externally retrieved value-hint block; let $y^\star$ be the correct SQL; and write $\bullet$ for one latent position. With $m = 2$ latent positions per role, evaluation runs

$$\underbrace{c}_{\text{prompt}} \;\|\; \mathsf{tag}(r_1) \underbrace{\bullet\bullet}_{m=2} \;\|\; \mathsf{tag}(r_2)\, \bullet\bullet \;\|\; \texttt{<|consensus|>} \;\|\; y^\star. \tag{5.1}$$

Only the final SQL is decoded. The two tags are inserted by the runner rather than generated, and the four latent positions emit no tokens. The explicit schema-link and candidate traces exist only as SFT supervision used by the curriculum; inference receives neither those texts nor the offline execution labels.

24 The curriculum: stage k

Definition 47 (Stage k)**.** At stage $k \in \{1, 2\}$, the first k roles are represented by latent states and each remaining role retains its explicit supervised teacher turn T_j:

$$c \;\|\; \mathsf{tag}(r_1)\, \bullet\bullet \;\|\; \cdots \;\|\; \mathsf{tag}(r_k)\, \bullet\bullet \;\|\; T_{k+1} \;\|\; \cdots \;\|\; T_2 \;\|\; \texttt{<|consensus|>} \;\|\; y^\star. \tag{5.2}$$

At stage 1, schema linking is latent while the candidate-generation trace remains explicit and supervised, so its offline execution-outcome examples contribute directly to the loss. At stage 2, that complete candidate turn is replaced by latent states and only the final correct SQL remains explicit. This is the Coconut curriculum transition that internalises the training-only failure examples rather than merely storing unused text in the records. The stage-2 checkpoint is passed to both GRPO arms.

```
stage 1: prompt | schema_linker oo | candidate_generator text | <|consensus|> | SQL
stage 2: prompt | schema_linker oo | candidate_generator oo | <|consensus|> | SQL
```

The current run performs one epoch at each stage over the full validated two-role corpus. Training uses one fresh process per batch, passes the stage explicitly with `--force-n-latent-roles`, carries optimiser state within a stage, and resets it at the stage boundary. The SQL-readiness auxiliary probe remains implemented for historical experiments but is disabled in this run (empty `SQL_READY_ROLE`, weight 0); with only two roles, the main final-SQL objective is the operative loss.

Part VI
Activation space: why any of this can work

There is a question that should be nagging by now. We are asking one set of weights to behave, in sequence, like six different specialists. Why should a single model be able to hold six distinct "personas" at all, rather than averaging them into one mush?

IMAD answers this question empirically, and the answer is the theoretical foundation this project rests on. This Part states it, and proves the one piece of mathematics it depends on.

25 The residual stream and the linear representation hypothesis

A transformer layer does not overwrite its input; it *adds* to it:

$$h^{(\ell+1)} = h^{(\ell)} + \mathrm{Attn}^{(\ell)}\big(h^{(\ell)}\big) + \mathrm{MLP}^{(\ell)}\big(h^{(\ell)}\big).$$

The running sum $h^{(\ell)}$ is called the **residual stream**. Because every layer contributes additively, the stream behaves like a shared vector workspace that layers read from and write to.

> **Linear representation hypothesis.** Many human-interpretable properties of a model's state — "this text is harmful", "this is code", "this is agent 3 speaking" — are encoded approximately as *directions* in the residual stream: there is a vector v such that the property's presence corresponds to a large value of $v \cdot h$.

This is a hypothesis, not a theorem. Its evidence is that interventions built on it work, which is what the next two sections are about.

26 Difference-in-means, and a proof that it is the right direction

Definition 48 (Difference-in-means)**.** Given activations from two classes at a fixed layer ℓ — positives $h_1^+, \dots, h_N^+$ and negatives $h_1^-, \dots, h_N^-$ — the **difference-in-means** direction is

$$d = \mu_+ - \mu_-, \qquad \mu_\pm = \frac{1}{N}\sum_{i=1}^{N} h_i^\pm, \qquad \hat{d} = \frac{d}{\|d\|}. \tag{6.1}$$

This is what `latentsql/steering/caa.py` computes, and it is IMAD's equation (3):

$$v_i = \frac{1}{|\mathcal{D}|}\sum_{(p,c)\in\mathcal{D}} \Big[h_\ell(p, c_i) - h_\ell(p, c_{\neg i})\Big],$$

where the context p is held *identical* and only the continuation varies: c_i is agent i's real response, $c_{\neg i}$ the average over the other agents' responses in the same slot.

Why the difference of means, out of every direction in a 3584-dimensional space? Because under one clean assumption it is provably optimal.

Assumption 49 (isotropic noise)**.** Model class-$+$ activations as $h = \mu_+ + \varepsilon$ and class-$-$ activations as $h = \mu_- + \varepsilon$, with the noise ε having the same spread in every direction:

$$\mathbb{E}\big[(w \cdot \varepsilon)^2\big] = \sigma^2 \quad \text{for every unit vector } w.$$

Lemma 50 (Cauchy–Schwarz)**.** *For any vectors a, b: $|a \cdot b| \le \|a\|\|b\|$, with equality exactly when a and b are parallel.*

Proof. For any real t, a squared length is non-negative:

$$0 \le \|a - tb\|^2 = \|a\|^2 - 2t(a \cdot b) + t^2\|b\|^2.$$

This is a quadratic in t that is never negative, so its discriminant is ≤ 0:

$$\big(2(a \cdot b)\big)^2 - 4\|a\|^2\|b\|^2 \le 0 \implies (a \cdot b)^2 \le \|a\|^2\|b\|^2.$$

Taking square roots gives the inequality. Equality forces $\|a - tb\|^2 = 0$ for some t, i.e. $a = tb$. □

Theorem 51 (Difference-in-means is the optimal linear probe under Assumption 49)**.** *Among all unit directions w, the one maximising Fisher's separation score*

$$J(w) = \frac{\big(w \cdot \mu_+ - w \cdot \mu_-\big)^2}{\mathrm{Var}(w \cdot h)}$$

is $w^\star = \hat{d} = (\mu_+ - \mu_-)/\|\mu_+ - \mu_-\|$.

Proof. By Assumption 49 the denominator equals σ^2 for *every* unit w; it is therefore a constant of the optimisation and can be dropped. The numerator is $(w \cdot d)^2$ with $d = \mu_+ - \mu_-$. By Lemma 50, for unit w,

$$|w \cdot d| \le \|w\|\,\|d\| = \|d\|,$$

with equality exactly when $w \parallel d$, i.e. $w = d/\|d\|$. Hence

$$\boxed{w^\star = \hat{d}}$$

maximises $(w \cdot d)^2$ and therefore $J(w)$. □

Remark 52 (what breaks without the assumption)**.** Real residual streams are not isotropic: some directions carry systematically more variance, and directions are correlated. Under a general shared covariance Σ the Fisher-optimal direction is

$$w^\star \propto \Sigma^{-1}(\mu_+ - \mu_-),$$

a whitened difference-in-means that down-weights noisy directions. Estimating and inverting a 3584×3584 covariance from a few hundred contrastive examples is ill-posed, which is the practical reason both this project's `caa.py` and the published steering work use the raw difference. This is a real gap between the theorem and the code, and it is stated rather than papered over.

27 Steering: using the direction

Definition 53 (Contrastive activation addition)**.** At inference, hook layer ℓ and add a multiple of the direction to its output:

$$h_\ell \;\longleftarrow\; h_\ell + \alpha\, v_i. \tag{6.2}$$

$\alpha > 0$ amplifies the property, $\alpha < 0$ suppresses it, $\alpha = 0$ is the unmodified model.

This is implemented as a forward hook in `caa.py::steer_layer`, at layer -8 (eight from the end) by default.

28 IMAD's finding, and what it licenses

IMAD trains a model to internalise a three-agent debate where each agent has a deliberately distinct style (chain-of-thought, self-critique, program-of-thought), extracts v_1, v_2, v_3 by (6.1), and then steers by (6.2) while measuring how closely the steered output resembles that agent's real responses.

Result. The internalised model is steerable toward each individual agent *substantially better than the base model is*: ROUGE-L AUC 0.802 $\rightarrow$ 0.903, a 12.7% improvement, with per-agent, per-metric gains from 6.1% to 25.0% and an average of 15.4%. At $\alpha = 0$ the two models are by construction identical, so the entire effect is attributable to internalisation having created structure that steering can find.

The interpretation, and the foundation of this project: internalising a multi-agent system does not blend the agents into an undifferentiated policy. It creates *agent subspaces* — separable directions in activation space, one per role. The agents survive internalisation as geometry.

That is the answer to the question this Part opened with. One model can in principle hold multiple specialists as different directions in a 3584-dimensional space. The current system asks for only two such specialisations, schema linking and candidate generation. It also explains why the role tags matter (Part V): the tag is what selects which subspace the following latent steps operate in.

IMAD tests the claim in the other direction too, which is a stronger test than the positive one: they deliberately instil a *malicious* agent through the same pipeline, then apply *negative* steering ($\alpha < 0$) to that agent's direction. Suppression is more effective after internalisation than on the base model, and costs less general capability — on GSM8K the internalised model stays stable across the full steering range while the base model's accuracy collapses at extreme coefficients. A behaviour that has been given its own direction is a behaviour you can find and remove.

29 What this does and does not establish for LatentSQL

Established	Not established
Internalised multi-agent training produces separable per-agent directions, in IMAD's setting, measured by steering.	That *our* two latent roles occupy separable subspaces. We have the extraction code and have not run the measurement on the two-role student.
Difference-in-means is the optimal linear probe under isotropic noise (Theorem 51).	That residual-stream noise is isotropic. It is not.
Structure tags improve subspace separation (IMAD, ablation).	The size of that effect for CHESS roles specifically.

Remark 54 (a use beyond interpretability)**.** If the two directions are real, steering toward `candidate_generator` with different α is a cheap diversity knob: it may recover some of CHASE-SQL's multi-path candidate generation from a single model at one forward pass per candidate. That is a stated motivation in `caa.py` and an untested idea, not a result.

Part VII
The training objective

30 Cross-entropy, from scratch

Training a language model means making the correct next token more probable. The standard way to say that numerically is **cross-entropy loss**.

Definition 55 (Per-token cross-entropy)**.** If the model assigns probability $p_t = p(x_{t+1} = x^\star_{t+1} \mid x_{\le t})$ to the correct next token, the loss at that position is

$$\mathcal{L}_t = -\log p_t.$$

Read it as a surprise meter. If the model was certain and right, $p_t = 1$ and $\mathcal{L}_t = -\log 1 = 0$: no surprise, no loss, nothing to learn. If it was certain and wrong, $p_t \to 0$ and $\mathcal{L}_t \to \infty$. If it had no idea at all among $|V|$ options, $p_t = 1/152064$ and $\mathcal{L}_t = \log 152064 \approx 11.93$.

Example 56 (the numbers to keep in mind)**.**

$\mathcal{L}_t$	p_t	interpretation
11.93	1/152064	uniform guess
1.0	0.368	plausible but unsure
0.1	0.905	confident
0.01	0.990	very confident
0.0008	0.9992	**memorised**

The last row is not hypothetical. It is the measured mean training loss at stage 4, and Part IX is about what it means.

Definition 57 (Masked sequence loss)**.** Given a label vector $z_1, \dots, z_T$ where $z_t = -100$ marks "do not supervise this position",

$$\mathcal{L} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} -\log p(x_{t+1} = z_t \mid \cdot), \qquad \mathcal{S} = \{t : z_t \neq -100\}. \tag{7.1}$$

The average is over supervised positions only.

> $|\mathcal{S}|$ — how many positions are graded — is the quantity to watch. It determines what the loss can and cannot see.

31 The COCONUT sequence loss, exactly as the code computes it

We can now write down the training loss precisely. Fix a training record: a prompt c, a gold SQL $y^\star$ with token ids $(g_1, \dots, g_n)$, and a stage k.

The function `_coconut_sequence_loss` (`latentsql/distill/latent_sft.py`) builds a list of input *embeddings* — not token ids, because latent positions have no token id — together with a parallel label list. In order:

1. **Prompt.** Append $e(c_1), \dots, e(c_{|c|})$, all labelled -100.
2. **For each role r_j, $j = 1, \dots, k$:**

(a) append the embeddings of $\texttt{tag}(r_j)$, labelled -100;

(b) repeat m times: run the model on everything built so far, take the last-layer last-position hidden state h, append $\text{align}(h)$ as the next input embedding, and append the label -100.

3. **Remaining explicit roles.** For each $j = k+1, \ldots, 2$, append the complete teacher turn T_j and supervise its tokens. At stage 1 this is the candidate-generation trace containing the labelled offline mutants and correct first-pass candidate; at stage 2 there is no explicit role turn left.

4. **Consensus.** Append the embeddings of `<|consensus|>`, labelled -100.

5. **Target.** Append $e(g_1), \ldots, e(g_n)$, labelled $g_1, \ldots, g_n$.

One final teacher-forced forward pass over the assembled sequence produces (7.1). Let U_k be the supervised token sequence consisting of the remaining explicit turns followed by $y^\star$, and write $u_i^{(k)}$ for its ith token. Writing Φ_k for the prompt plus the first k latent roles, the implemented main loss is

$$\boxed{\mathcal{L}_k^{\text{main}} = -\frac{1}{|U_k|} \sum_{i=1}^{|U_k|} \log p_\theta\big(u_i^{(k)} \,\big|\, \Phi_k,\ u_{<i}^{(k)}\big)} \tag{7.2}$$

At stage 1, $|\mathcal{S}|$ includes the candidate-generation trace and the final SQL; a one-record verification of the corrected loss logged 252 supervised tokens rather than the SQL-only count. At stage 2, $|\mathcal{S}| = n$ because both roles are latent and only the final SQL remains explicit. The prompt and latent positions are never graded directly. This shrinking supervised suffix is the mechanism by which the explicit failure examples are first learned and then removed.

31.1 Why the latent positions get gradients anyway

A natural objection: if latent positions are not supervised, how does anything about them get trained?

Through the chain rule. Every supervised suffix token depends on the latent prefix, and each latent vector is a differentiable function of the model's parameters (it is align of a hidden state that the model produced). So

$$\frac{\partial \mathcal{L}_k^{\text{main}}}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}}{\partial \Phi_k}}_{\text{how the prefix should change}} \cdot \underbrace{\frac{\partial \Phi_k}{\partial \theta}}_{\text{through all } km \text{ latent steps}} + \text{direct paths.}$$

Nothing forces the latent vectors to reproduce a unique verbal explanation. At stage 1 they are shaped by the pressure to predict the explicit candidate trace and final SQL; at stage 2 they are shaped by the final SQL after the candidate trace has been removed. This is Coconut's own framing: continuous thoughts are trained to facilitate the remaining reasoning and answer, not to encode a word-for-word hidden transcript.

Remark 58 (training does $km + 1$ forward passes)**.** The latent steps are sequential — each needs the previous one's output as its input — so a stage-k record costs km forward passes to build the prefix plus one more for the loss. In the current final stage, $k = 2$ and $m = 2$, so that is five forward passes. This is still slower than ordinary teacher forcing, and the code uses `use_cache=False` in this path for the memory-safer shape at the cost of recomputation.

32 The optional SQL-readiness probe (disabled in the current run)

The **SQL-readiness probe** is an optional auxiliary term that grades SQL at an earlier point. It was designed for the historical SQL-direct curriculum, where later role turns were dropped rather than retained as Coconut supervision. It is disabled in the current run, and the propositions below describe that historical objective rather than the active two-role loss.

Definition 59 (The probe)**.** Fix a role index $j^\star$ (the parameter `SQL_READY_ROLE`) and a token budget κ (`SQL_READY_TOKENS`). During the loop of step 2 above, immediately after finishing role $j^\star$'s latent steps, build a *second*, temporary sequence:

$$\Phi_{j^\star} \,\|\, \texttt{<|consensus|>} \,\|\, (g_1, \ldots, g_\kappa),$$

supervise only the κ target tokens, and record its loss

$$\mathcal{L}^{\text{probe}} = -\frac{1}{\kappa} \sum_{i=1}^{\kappa} \log p_\theta\big(g_i \mid \Phi_{j^\star}, g_{<i}\big). \tag{7.3}$$

The total objective is

$$\boxed{\mathcal{L}_k = \mathcal{L}_k^{\text{main}} + \lambda\, \mathcal{L}^{\text{probe}}} \tag{7.4}$$

where λ is `SQL_READY_WEIGHT`. In the current two-role experiment $\lambda = 0$, so $\mathcal{L}_k = \mathcal{L}_k^{\text{main}}$; the probe remains documented because it was active in the historical six-role runs analysed in Part IX.

The intent, in one sentence: *teach the model that* `<|consensus|>` *means "start writing SQL now" regardless of how many latent roles preceded it.*

32.1 A degeneracy: why $j^\star = k$ does nothing

Proposition 60. *If $j^\star = k$ — the probe fires after the* last *latent role of the current stage — then $\mathcal{L}^{\text{probe}}$ is a truncated copy of $\mathcal{L}_k^{\text{main}}$, and (7.4) reduces to a reweighting of the main loss rather than adding any new constraint.*

Proof. The probe's prefix is $\Phi_{j^\star}$ followed by `<|consensus|>`. When $j^\star = k$, $\Phi_{j^\star} = \Phi_k$, which is exactly the main sequence's prefix; the main sequence also continues with `<|consensus|>`. The two sequences are therefore token-for-token and vector-for-vector identical up to that point. The probe then supervises $(g_1, \ldots, g_\kappa)$; the main loss supervises $(g_1, \ldots, g_n)$ with $\kappa \le n$. So $\mathcal{L}^{\text{probe}}$ is the mean of the first κ of the same n per-token terms that $\mathcal{L}_k^{\text{main}}$ averages, computed at the same positions from the same states. It contributes no gradient direction that $\mathcal{L}_k^{\text{main}}$ does not already contain. □

Visually, in the historical six-role experiment at stage 4 with $j^\star = 4$:

```
probe :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL[:16]
main  :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL
         ^--------------- identical ---------------^
```

32.2 Why $j^\star = k - 1$ is not degenerate

In that historical experiment, set $j^\star = 3$. At stage 4:

```
probe :  prompt | r1 oo | r2 oo | r3 oo |         <|consensus|> | SQL[:24]
main  :  prompt | r1 oo | r2 oo | r3 oo | r4 oo | <|consensus|> | SQL
                                       ^ probe cuts in one role early
```

Now the probe's prefix is a sequence the main loss never evaluates. In one optimisation step the model is graded on producing the same SQL from *two different prefix lengths*. It cannot satisfy both by memorising one positional pattern; the only representation that satisfies both is the general rule "`<|consensus|>` means SQL, whatever preceded it".

There is a second reason to prefer $j^\star = 3$: it is stage-independent. The probe is active only when $k \geq j^\star$ (the `sql_ready_active` gate in the code), so

$j^\star$	stage 4	stage 5	stage 6
4	degenerate	mid-prefix	mid-prefix
3	mid-prefix	mid-prefix	mid-prefix

With $j^\star = 4$ the probe was a genuine constraint at exactly the two stages that were not broken, and a no-op at the one that was.

Remark 61 (honest note, carried forward)**.** Proposition 60 shows $j^\star = 4$ was the wrong setting. It does *not* show that fixing it is what repaired stage 4. Measured values of $\mathcal{L}^{\text{probe}}$ during the retrain were of order 10^{-4} (Table 1) — as saturated as the main loss — so its gradient contribution was tiny. This is picked up in Section 44.

Part VIII
The optimiser, and one uncomfortable fact about Adam

The loss of Part VII has to be turned into weight updates. How that is done turns out to matter as much as the loss itself, so it gets its own Part.

33 What is actually being trained: LoRA

We do not update all 7×10^9 parameters. We use **LoRA** (Low-Rank Adaptation): for a selected weight matrix $W_0 \in \mathbb{R}^{a\times b}$, freeze it and learn a low-rank correction,

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{a\times r},\ A \in \mathbb{R}^{r\times b},\ r \ll \min(a,b). \tag{8.1}$$

Only A and B are trainable. Our settings are $r = 32$, $\alpha = 64$ (so the scaling factor is $\alpha/r = 2$), applied to `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj` in every layer.

Remark 62. LoRA matters here for a reason beyond memory: because W_0 is frozen, whatever the base model already believed is still fully present. It has not been overwritten, only offset. Mechanism 3 of Part IX is exactly a case of the frozen base's prior reasserting itself.

34 Adam, derived

Plain gradient descent takes $\theta \leftarrow \theta - \eta g$ with $g = \nabla_\theta \mathcal{L}$. Its weakness is that a single global η must suit parameters whose gradients differ in scale by orders of magnitude. Adam fixes this by giving each parameter its own effective step size, estimated from its own gradient history.

Definition 63 (Adam)**.** With $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and moments initialised to zero, at step t:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t && \text{(momentum: a running mean of } g\text{)} \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 && \text{(a running mean of } g^2\text{)} \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} && \text{(bias correction)} \\ \theta_t &= \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \end{aligned} \tag{8.2}$$

The bias correction exists because $m_0 = v_0 = 0$ drags the early averages toward zero; dividing by $1 - \beta^t$ removes that bias exactly.

34.1 The normalisation, and what it costs

Look at the update direction in (8.2):

$$\Delta\theta = -\eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

$\hat{m}_t$ is an average of g; $\sqrt{\hat{v}_t}$ is (roughly) the root-mean-square of g. Their ratio is *scale-free*: multiply every gradient by 1000 and both numerator and denominator scale by 1000, leaving $\Delta\theta$ unchanged.

Adam does not take smaller steps when the gradient is small. It takes steps of size set by η, in the direction the gradient points, essentially regardless of the gradient's magnitude. A gradient of 10^{-4} moves the weights about as far as a gradient of 10^{-1}.

This is a feature when the small gradient is a genuine, consistent signal. It is a hazard when the small gradient is *noise* — which is exactly the situation when the loss has saturated.

Proposition 64 (the first step after a moment reset is a pure sign step)**.** *Immediately after the moments are reset to zero, the first Adam update is*

$$\Delta\theta = -\eta \, \frac{g}{|g| + \varepsilon} \;\approx\; -\eta \, \mathrm{sign}(g),$$

independently of $\|g\|$.

Proof. Set $t = 1$ with $m_0 = v_0 = 0$. From (8.2),

$$m_1 = (1 - \beta_1)g, \qquad v_1 = (1 - \beta_2)g^2.$$

Bias correction divides by $1 - \beta_1^1 = 1 - \beta_1$ and $1 - \beta_2^1 = 1 - \beta_2$ respectively, which cancels those factors exactly:

$$\hat{m}_1 = \frac{(1 - \beta_1)g}{1 - \beta_1} = g, \qquad \hat{v}_1 = \frac{(1 - \beta_2)g^2}{1 - \beta_2} = g^2.$$

Hence $\sqrt{\hat{v}_1} = |g|$ and

$$\Delta\theta = -\eta \, \frac{g}{|g| + \varepsilon},$$

which for $|g| \gg \varepsilon$ is $-\eta \, \mathrm{sign}(g)$: a step of length exactly η per coordinate, whatever the gradient's size. □

Combine Proposition 64 with the curriculum's design choice of resetting the optimiser at every stage boundary (Section 24) and the consequence is immediate: **the first update of every stage is a full-size step in whatever direction the residual gradient happens to point**, even if that gradient is pure noise from an already-memorised objective.

35 Gradient accumulation, and how few steps there really are

Gradients are accumulated over several records before an update:

```
(loss / cfg.grad_accum).backward()
...
if n_acc % cfg.grad_accum == 0:
    torch.nn.utils.clip_grad_norm_(params, 1.0)
    optim.step()
    optim.zero_grad(set_to_none=True)
```

This is mathematically the gradient of the mean loss over `grad_accum` records — a larger effective batch at constant memory.

The runner sets `--grad-accum "$limit"`, i.e. equal to the batch size. So one process = one batch = *exactly one* optimiser step.

Example 65 (the real step budget)**.** With $N = 94$ usable records, `BATCH_SIZE` $= 16$ and `EPOCHS_PER_STAGE` $= 3$:

$$\left\lceil \frac{94}{16} \right\rceil = 6 \text{ batches per epoch}, \qquad 6 \times 3 = \boxed{18 \text{ optimiser steps per stage}}.$$

Meanwhile $94 \times 3 = 282$ forward/backward passes are logged per stage, which matches the step-record counts exactly.

18 steps is very few. Under Proposition 64 the first of them is a full-η sign step, and with $\beta_1 = 0.9$ the momentum is still far from equilibrium for several more. At $\eta = 5 \times 10^{-5}$ and LoRA's $\alpha/r = 2$ amplification, this is real movement in the adapter, in a direction chosen by whichever one or two of 16 records still had non-trivial loss.

Gradient clipping to norm 1.0 is applied before every step. Note what it does and does not do: it bounds the update when gradients are *large*. It has no effect at all when they are small, which is the regime that causes trouble here.

Part IX
A historical worked failure: the obsolete six-role stage-4 collapse

This Part records a real failure of the earlier six-role curriculum. It is retained because it motivated the safer optimiser handling and diagnostics used now, but its accuracy numbers are not results for the current two-role architecture. The current experiment has only stages 1 and 2, disables the SQL-readiness probe, and cannot have a stage-4 collapse by construction.

36 The symptom

The obsolete CHESS-step curriculum was run for six stages. After each stage the checkpoint was probed on a fixed 30-question held-out sample.

stage k	train loss (mean)	probe EX (%)	avg output tokens
1	0.1332	66.67	62.7
2	0.0186	66.67	67.8
3	0.0019	63.33	61.0
4	**0.0008**	**26.67**	**125.7**
5	0.0015	56.67	72.4
6	0.0014	63.33	67.4

Stage 4 loses 36.66 points of execution accuracy and doubles its output length.

The project's own earlier documentation described this as "an optimization/representation instability". That was wrong, and the table above is the refutation: **stage 4 has the lowest mean loss and the lowest maximum loss of any stage** (max = 0.1265, versus 0.9836 at stage 1). There is no instability. Training went better at stage 4 than anywhere else.

37 What the failures actually looked like

14 of the 30 stage-4 predictions were unparseable, and all 14 failed the same way — schema-linking prose after the consensus marker, repeated until the 200-token generation cap:

```
<|consensus|>Tables:
Tables: visitor, visit, museum
Tables:
Tables: visitor, visit, museum
... (to the cap)
```

The doubled output length is the same fact seen from the other side: those are not longer queries, they are cap-length loops.

The model has not forgotten SQL. It has forgotten that `<|consensus|>` means "emit SQL". It has reverted to a different, older belief about what follows a role tag.

38 Mechanism 1: the objective is saturated by stage 3

In the historical implementation, $|\mathcal{S}| = n \approx 62$ because only the final SQL was supervised; it used $N = 94$ records and 3 epochs per stage. Equation (7.2) now describes the corrected explicit-suffix curriculum and should not be read as a description of this old run.

By stage 3 the mean per-token cross-entropy is 0.0019; by stage 4 it is 0.0008, i.e. $p_t \approx 0.9992$ on the correct next token.

94 SQL strings, seen 3 times each per stage across six stages, at a per-token loss of 8×10^{-4}. They are **memorised**. There is essentially no learning signal left in $\mathcal{L}^{\text{main}}$.

The reason this happened so completely was the old SQL-direct cut. The code at the time was:

```
add_text(chat, supervise=False)
for idx, role in enumerate(cfg.roles):
    if role == "consensus":
        break
    if idx >= n_latent_roles:
        break            # <-- latent_sft.py:113
    ...
add_text(T_CONSENSUS_OPEN, supervise=False)
add_token_ids(target_ids, supervise=True)
```

That historical `break` removed every role past the current stage instead of appending it as supervised text. Confirmation from the old logs is that `text_loss` is `null` in all 1908 step records. The corrected implementation now has the missing explicit-role branch; stage 1 supervises the candidate turn and stage 2 replaces it with latent states.

This is where LatentSQL departs from Coconut. Coconut replaces the first k reasoning steps with latents *and keeps the remaining steps as supervised text*, so the supervision set shrinks gradually as k grows. Here the remaining steps are deleted, so the supervision set is a constant ≈ 62 SQL tokens at every stage. The curriculum's difficulty ramps; the objective's difficulty does not.

39 Mechanism 2: Adam steps into the noise at full size

Now apply Part VIII. Per Example 65 there are 18 optimiser steps per stage, and the runner deletes `optim_state.pt` at each stage boundary, so every stage begins from zero momentum.

By Proposition 64, the first step of stage 4 is

$$\Delta\theta = -\eta \operatorname{sign}(g), \qquad \eta = 5 \times 10^{-5},$$

a full-magnitude step regardless of the fact that $\|g\|$ is now tiny. And by the scale-freeness of (8.2), the following steps are not much smaller.

The two mechanisms compose badly. Mechanism 1 says the gradient carries almost no information. Mechanism 2 says Adam will nonetheless move the weights the full distance η in whatever direction that near-zero gradient points. The adapter drifts, driven by whichever one or two of 16 records in a batch still have non-trivial loss.

40 Mechanism 3: where the drift lands

Drift alone would be harmless if the weights had nowhere bad to go. They do.

The curriculum starts from `sft_7b_bf16`, an adapter trained by ordinary SFT on *full written debate transcripts* — schema-linking blocks that literally begin `Tables: ...`, candidate discussions, unit tests. In those transcripts, the thing that follows a role tag is prose.

By (8.1) that belief was never deleted: W_0 is frozen and the LoRA correction merely offsets it. The latent curriculum trains a competing behaviour — "after the tags, emit SQL" — but only on the 94 training records, where the model already gets it right and therefore is never penalised for the old prior. On any held-out question the two behaviours compete, and a few full-size Adam steps in a noise direction are enough to tip the balance.

> This is a **generalisation** failure, not an optimisation one, and it is invisible to the training objective by construction. The objective is evaluated on exactly the distribution where the failure does not occur.

Proposition 66 (the failure is unobservable to the objective)**.** *Let $\mathcal{D}_{\text{tr}}$ be the 94 training records and $\mathcal{D}_{\text{ho}}$ the held-out set. Suppose parameters θ satisfy $\mathcal{L}(\theta;\, x) < \epsilon$ for all $x \in \mathcal{D}_{\text{tr}}$ while the model's greedy continuation after `</consensus/>` on some $x' \in \mathcal{D}_{\text{ho}}$ is not SQL. Then no quantity computed from $\{\mathcal{L}(\theta; x)\}_{x \in \mathcal{D}_{\text{tr}}}$ — its mean, maximum, variance, or trajectory — can distinguish θ from a θ' that generalises correctly.*

Proof. Immediate: the hypothesis fixes the values of every such quantity on $\mathcal{D}_{\text{tr}}$ to be within ϵ of the same numbers for both θ and θ', while the behaviour that differs is evaluated only on $\mathcal{D}_{\text{ho}}$, which those quantities do not depend on. □

This is why the per-stage held-out probe exists in the runner at all, and why it caught something a loss curve never would.

41 The bonus finding: the fix that was already configured did nothing

The run had `SQL_READY_ROLE=4` set, precisely to anchor the "consensus means SQL" rule. By Proposition 60, at stage 4 with $j^\star = 4$ the probe is a truncated duplicate of the main loss.

> The auxiliary constraint was a genuine mid-prefix anchor at stages 5 and 6 — which were never the problem — and a no-op at stage 4, which was.

42 The repair

Two changes, applied from stage 4 onward, resuming from the stage-3 checkpoint into a fresh run directory (`chess_coconut_sql_direct_s4fix`) so the original run and the downstream GRPO adapter were left untouched:

knob	before	after	targets
learning rate, stage ≥ 4	5×10^{-5}	1×10^{-5}	mechanism 2
`SQL_READY_ROLE`	4	3	mechanism 3
`SQL_READY_WEIGHT`	0.5	0.5	—
`SQL_READY_TOKENS`	16	24	—

The learning-rate change reduces the size of every drift step by 5×. The probe change makes (7.4) a real constraint at stages 4, 5 and 6 (Section 32.1).

Remark 67 (what was deliberately not changed)**.** The obvious structural fix — restore Coconut's remaining-steps-as-text supervision, so that $|\mathcal{S}|$ grows with the curriculum and the objective never saturates — was *not* attempted. It changes the format the downstream GRPO stage and the evaluation harness both assume. It is recorded as the next lever if a hyperparameter fix had failed.

43 Result of the retrain

stage	EX old	EX new	unparseable	hit 200-cap	out tokens
4	26.67	**66.67**	14 → **0**	14 → **0**	125.7 → **61.6**
5	56.67	**66.67**	3 → **0**	2 → **0**	72.4 → **61.2**
6	63.33	63.33	0 → 0	1 → **0**	67.4 → **62.2**

The collapse is gone (+40.00 EX at stage 4) and the `Tables:` loop is eliminated outright. Note also what the table reveals about the *original* run: stage 5's apparent "recovery" was never complete — it still carried 3 unparseable predictions and a 72.4-token average. Stages 5 and 6 had been partly spending their capacity undoing stage 4's damage.

On the full held-out set:

run	n	EX	bird	spider	out tok	unparseable	latency
stage 6, original	90	52.22	33.33	71.11	67.5	4	2.212 s
stage 6, **retrained**	90	**57.78**	**37.78**	**77.78**	**61.2**	**0**	**2.073** s

+5.56 EX, improving on *both* benchmarks, output length down 9.3%, latency down 6.3%, every unparseable prediction eliminated.

Remark 68 (probe optimism)**.** The $n = 30$ stage-6 probe is flat between the two runs (63.33 both times); the advantage appears only on the full $n = 90$ set. A 30-question probe has a standard error of roughly 9 points, so it can rank two nearby checkpoints incorrectly. It is a smoke detector, not a scoreboard.

44 Training loss during the retrain, and the attribution caveat

stage	steps	$\mathcal{L}^{\text{main}}$ mean	$\mathcal{L}^{\text{main}}$ max	$\mathcal{L}^{\text{probe}}$ mean
4	311	0.00030	0.0252	2.39×10^{-4}
5	282	0.00118	0.3388	1.44×10^{-3}
6	282	0.00003	0.0001	8.15×10^{-6}

Table 1: Retrained stages. Both loss terms are saturated throughout.

Two knobs moved at once. This is not a clean ablation.

The evidence favours the learning rate doing nearly all the work: $\mathcal{L}^{\text{probe}}$ logged $\sim 10^{-4}$ throughout, i.e. the auxiliary term was as saturated as the main one and contributed almost no gradient.

Supported claim: lowering the late-stage learning rate fixes the stage-4 collapse.
Not supported: the SQL-readiness probe fixes it.

A clean separation requires a third run at $\eta = 10^{-5}$ with `SQL_READY_WEIGHT`$= 0$. It has not been run.

Note also that Table 1 independently confirms Mechanism 1: at the *lower* learning rate the losses are, if anything, even smaller. A saturated objective was never the thing the learning rate fixed — what it fixed was how far the weights travelled while the objective was blind.

45 Two infrastructure failures worth recording

Both occurred at a batch boundary, and both trace to one fact: *a process exiting does not mean its CUDA context has been released.*

1. **Out of memory.** Two per-batch processes overlapped (17.47 + 5.91 GB against a 22.57 GB budget) and the run aborted — even though the batch's own peak was only 18.5 GB. Fix: a `gpu_settle` helper that polls `nvidia-smi` until the card has drained below 1500 MiB before each batch and each evaluation.

2. **A silent CPU downgrade, which was worse.** The next attempt logged *zero* training steps in 48 minutes while burning 1h53m of CPU time at 18.3 GB resident, with the GPU idle at 40 MiB. `torch.cuda.is_available()` had returned `False` transiently, `resolve_device("auto")` therefore returned `"cpu"`, and a 7B model in `bfloat16` trained happily in host RAM roughly 500× too slowly *without raising anything.*

An out-of-memory error at least fails loudly. A silent device downgrade produces a "successful" run, exit code 0, days later, with a checkpoint trained on nothing. The fix is a guard that converts the silent fallback into an immediate error, shown below; the runner sets `LATENTSQL_REQUIRE_CUDA=1` on every batch and every evaluation, and retries each batch once after 20 s, since both failure modes clear within seconds.

```
if torch.cuda.is_available():
    return "cuda"
if os.environ.get("LATENTSQL_REQUIRE_CUDA", "").strip() not in ("", "0", "false", "False"):
    raise RuntimeError(
        "LATENTSQL_REQUIRE_CUDA is set but torch.cuda.is_available() is False - "
        "refusing to fall back to CPU.")
```

Part X
Reinforcement learning: internalisation with GRPO

Curriculum SFT (Parts V–VII) trains the explicit candidate trace before replacing it with latent states. Its corpus is filtered by execution and its mutant labels are produced by execution, but its differentiable objective is still token cross-entropy rather than an execution reward. It also does not reward brevity. GRPO supplies those sequence-level behavioural signals.

46 Group Relative Policy Optimisation, from the policy-gradient theorem

Definition 69 (The objective)**.** Let $\pi_\theta(y \mid x)$ be the probability the current model assigns to generating completion y given input x. Training wants to maximise expected reward,

$$J(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[R(x,y)\big] = \sum_y \pi_\theta(y \mid x)\, R(x,y).$$

Theorem 70 (Policy gradient)**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{y\sim\pi_\theta}\big[\nabla_\theta \log \pi_\theta(y \mid x)\cdot R(x,y)\big].$$

Proof. Since R does not depend on θ, $\nabla_\theta J = \sum_y \nabla_\theta \pi_\theta(y \mid x)\, R(x,y)$. Using the log-derivative identity $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$ (chain rule on log), this becomes $\sum_y \pi_\theta(y \mid x)\, \nabla_\theta \log \pi_\theta(y \mid x)\, R(x,y)$, which is by definition an expectation under π_θ. □

This is what makes RL on a language model tractable at all: the intractable sum over every possible SQL query is replaced by sampling a handful of actual completions and averaging.

Proposition 71 (a baseline is free)**.** *For any b not depending on y, $\mathbb{E}_{y\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(y \mid x)\cdot b] = 0$.*

Proof. The left side equals $b \cdot \nabla_\theta \sum_y \pi_\theta(y \mid x)$. But $\sum_y \pi_\theta(y \mid x) = 1$ identically (it is a probability distribution), and the derivative of a constant is 0. □

So subtracting any y-independent quantity leaves the expected gradient unchanged while reducing its variance — which licenses GRPO's construction.

Definition 72 (GRPO advantage)**.** For a group of G rollouts $y_1, \ldots, y_G$ sampled at the same x, with $r_i = R(x, y_i)$,

$$A_i = \frac{r_i - \bar{r}}{s}, \qquad \bar{r} = \text{mean}(r_1, \ldots, r_G), \quad s = \text{std}(r_1, \ldots, r_G). \tag{9.1}$$

Corollary 73 (a saturated group has zero gradient)**.** *If $r_1 = \cdots = r_G$, then $A_i = 0$ for every i, and the resulting gradient estimate $\frac{1}{G}\sum_i A_i \nabla_\theta \log \pi_\theta(y_i \mid x)$ is exactly the zero vector.*

Proof. $\bar{r} = r_1$ makes every numerator $r_i - \bar{r} = 0$, so every $A_i = 0$ regardless of how $s = 0$ is handled. □

> This is not a hand-wave — it is exactly what `grpo.py` logs as `adv_nonzero_frac` $= 0$: a group where every sampled rollout agrees on reward (all correct, or all wrong) contributes *nothing* to that step. It is the single most common failure mode this project's own diagnostics recorded for GRPO.

47 The clipped surrogate and the KL penalty

The code (`grpo.py`) does not apply (9.1) directly to $\log \pi_\theta$; it uses PPO's clipped surrogate, with importance ratio $\rho = \exp(\log \pi_\theta(y \mid x) - \log \pi_{\theta_{\text{old}}}(y \mid x))$ (here $= 1$ on the very step it is sampled from, since $\theta = \theta_{\text{old}}$ at that instant):

$$\mathcal{L}_i^{\text{PG}} = -\min\big(\rho_i A_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\, A_i\big), \qquad \text{KL} \approx e^{\delta} - \delta - 1, \quad \delta = \log \pi_\theta - \log \pi_{\text{ref}},$$

$$\mathcal{L} = \mathcal{L}^{\text{PG}} + \beta_{\text{KL}} \cdot \text{KL}, \qquad \epsilon = 0.2.$$

The clip bounds how far one step can trust a single sample. The general implementation contains the non-negative KL estimator shown above, but the latent loss path does not yet compute a valid frozen latent-prefix reference policy. The current configuration therefore sets $\beta_{\text{KL}} = 0$ explicitly for both reward arms instead of reporting a regulariser that is not active. Historical non-latent runs used $\beta_{\text{KL}} = 0.02$.

48 IMAD's reward, specialised to SQL execution

IMAD's reward (its eq. 1) is

$$r(x, y) = w_{\text{fmt}}\, R_{\text{fmt}} + w_{\text{clip}}\, R(y; l), \qquad R(y; l) = \begin{cases} 1 & y^\star \text{ appears in the first } l \text{ tokens of } y \\ 0 & \text{else.} \end{cases} \tag{9.2}$$

Two schedules anneal over training:

$$w_{\text{fmt}} : 1.0 \to 0.05, \qquad l : l_0 \to l^\star.$$

The mechanism, in one sentence: the decaying format reward removes the incentive to *verbalise* the debate; the shrinking length limit makes verbalising it *impossible* while still earning the correctness reward. The only strategy left is to do the multi-perspective analysis internally and answer directly.

For Text-to-SQL, "correct" is redefined as execution match (Definition 1) rather than string containment, so $R(y; l)$ actually runs the clipped SQL against the database. Two shaping terms are added, small so IMAD's two terms still dominate: $w_{\text{exec_ok}}$ (the query parses and runs) and $w_{\text{tag_order}}$ (structure tags in the canonical order).

48.1 Dense partial credit, for when GRPO would otherwise learn nothing

Corollary 73 is not hypothetical here: a rollout group that is *entirely wrong* has zero variance under (9.2), and by Proposition 71/Corollary 73 contributes no gradient. Two extra terms, awarded *only to incorrect rollouts*, break the tie between a near-miss and a badly-wrong one:

$$w_{\text{atomic}} \cdot R_{\text{atomic}}(y, y^\star) \ + \ w_{\text{memory}} \cdot R_{\text{memory}}(y)$$

where R_{atomic} scores atomic-SQL-operator overlap with the gold query and R_{memory} scores similarity to previously-verified reasoning traces (annealed on w_{fmt}'s schedule, so it fades exactly as the incentive to reason in prose fades).

48.2 Brevity, for when a group is entirely correct

The mirror problem: a group where every rollout is already correct *also* has zero variance, and — since the entire purpose of internalisation is to stop verbalising — this is precisely the case where GRPO most needs to say something. Among correct rollouts only,

$$R_{\text{brevity}}(y) = \max\Big(0,\ 1 - \frac{|y|}{\text{ref}}\Big),$$

so a terse correct answer outscores a verbose correct answer, giving the group non-zero variance again and restoring a gradient.

48.3 Spurious correctness: when a group is entirely correct, but shouldn't be

Brevity fixes one way an all-correct group can be a trap: every rollout is right, so there is nothing to teach from. There is a second, nastier way the *same* group can look all-correct without actually being so — and this one is not about length at all.

Step 1: the problem, with the smallest possible example. Definition 1 checks one thing: does the predicted query return the same rows as gold, *on the one database the benchmark ships*. That database is a single, fixed snapshot of the data. Two queries that mean completely different things can still agree on it, purely by accident of what values happen to be sitting in the table that day.

Example 74 (two different queries, one lucky database)**.**

```
Question:   Which employees work in the Research department?
Gold:       SELECT name FROM employee WHERE dept = 'Research';
Candidate:  SELECT name FROM employee WHERE salary BETWEEN 50000 AND 60000;
```

Suppose, on today's data, every Research employee happens to earn between $50,000 and $60,000, and nobody else does. Both queries return the same two names. Definition 1 scores the candidate 1 — correct. It is not correct: it says nothing about departments at all, and would return the wrong names the moment a new employee is added with a salary in that range but a different job.

Why this is an RL problem, not just an eval problem. In GRPO, the reward *is* R_{clip} (eq. 9.2). If a whole group of G rollouts each independently stumbles onto a different accidentally-matching query, the group looks exactly like the saturated, all-correct case Brevity was built for (Corollary 73: zero variance, nothing to learn) — except here the group isn't actually all correct. Left alone, GRPO has no way to tell a robustly-correct rollout from a lucky one, and nothing stops it from reinforcing the lucky kind just as happily as the real kind.

Step 2: why offline robustness checks do not directly transfer. A stronger offline evaluation can expose Example 74 by building several modified copies of the database and requiring gold and the prediction to agree on *every* copy, not just the original. One copy can add a Research employee earning outside $50,000–$60,000, while another adds a non-Research employee earning inside that range; the candidate fails both and is correctly exposed as wrong.

An offline evaluator may choose which row to add after seeing both the gold and candidate queries. That does not transfer directly to a reward function: GRPO evaluates a brand-new, unforeseen rollout at every training step, so there is no fixed query pair for which a row can be designed in advance. Only the *gold* query is known ahead of time. The principle — check agreement across several databases rather than one — transfers, but instance construction must be automatic and depend only on gold.

Step 3: three automatic ways to build those extra databases. All three depend on the gold query and database, not on an LLM or a particular predicted query. Instances are cached by database path and gold SQL, shared across rollouts and arms, and deleted after use when cleanup is enabled.

Method A — plant boundary and join counterexamples. Parse gold's `WHERE` and `HAVING` predicates and its `JOIN ... ON` equalities. Equality predicates receive a satisfying row and a near-miss row. Numeric inequalities receive rows at the threshold and on the appropriate $\pm\varepsilon$ side; `BETWEEN` receives rows at and just outside both bounds. Values for other same-table conjuncts are chosen jointly so the synthetic row satisfies those filters rather than accidentally failing them. Equi-joins receive a dangling unmatched row to distinguish inner from left joins and, where constraints allow, a duplicate-key row to expose fan-out or missing `DISTINCT`.

Method B — keep only instances that test a plausible error. Generate small one-edit mutants of gold: flip a comparison, nudge a numeric literal, swap `AND`/`OR`, or change inner and left join types. Keep a targeted instance only when gold disagrees with at least one executable mutant on it. This asks "does this database separate gold from a plausible nearby query?" deterministically.

Method C — shake what already exists. As an untargeted fallback, copy the database, then drop, duplicate or numerically jitter a small sample of existing rows. Identity and foreign-key columns are excluded from jitter, and integer primary keys are omitted when duplicating so SQLite can assign fresh keys.

Example 75 (what Method C can and cannot catch)**.** Suppose gold correctly counts distinct singers at one stadium, `COUNT(DISTINCT singer_id)`, and the candidate forgot `DISTINCT`, `COUNT(singer_id)`. If no singer in today's data happens to perform there twice, the two counts agree — another spurious match. **Duplicating one existing link row** breaks this: gold's `DISTINCT` still counts that singer once; the candidate now counts them twice. Caught, with no idea in advance that "missing `DISTINCT`" was the risk.

Now take Example 74 instead. Every row that exists today already satisfies *both* filters or *neither* — there is no existing Research-but-low-salary row to duplicate or delete, because it never existed in the data to begin with. Reshuffling existing rows cannot reveal the two filters are different; only *inserting* the missing row (Method A) can. **Method C is good at multiplicity bugs — duplicate rows, a missing `DISTINCT`, a join that fans out — and blind to filter/threshold bugs**, which is exactly why it is the fallback, not the main method: used only when `sqlglot` is unavailable, or gold has no `WHERE`/`HAVING` literal for Method A to anchor on at all (a bare `SELECT COUNT(*) FROM t`).

Step 4: turning agreement-across-instances into a reward term. Let $\mathcal{I}(y^\star)$ be the set of stress instances built from gold $y^\star$ by Methods A–C. The GRPO arm uses up to three instances

for training cost, while the final true-EX audit uses up to five. Define

$$R_{\text{spurious}}(y) \;=\; \frac{1}{|\mathcal{I}(y^\star)|} \sum_{D \in \mathcal{I}(y^\star)} \mathbb{1}\big[\text{exec}(y, D) = \text{exec}(y^\star, D)\big], \tag{9.3}$$

the fraction of stress instances on which the prediction still agrees with gold — a dense score in $[0, 1]$, not a binary pass/fail, exactly like the other two shaping terms in this Part. Awarded, like brevity, *only* to rollouts that already passed the original check ($R_{\text{clip}} = 1$): a rollout that is wrong on the shipped database needs no further test, it is already being handled by §48.1's dense partial credit. The total reward (extending eq. 9.2) gains one more term,

$$r(x, y) \mathrel{+}= \; -w_{\text{spurious}} \cdot \big(1 - R_{\text{spurious}}(y)\big), \qquad \text{computed only when } R_{\text{clip}}(y) = 1, \tag{9.4}$$

a penalty that is exactly 0 when the prediction survives every stress instance (fully robust — no change in behaviour from before this section) and grows toward $-w_{\text{spurious}}$ the more instances expose it as lucky rather than correct.

What this buys, stated precisely. It does not change what counts as correct on the original database — R_{clip} is untouched. It changes what a GRPO group looks like *after* all of today's queries are already right under R_{clip}: a group that used to be uniformly 1.0 (dead, by Corollary 73) can now split into genuinely robust rollouts and spuriously-lucky ones, restoring exactly the kind of within-group disagreement GRPO needs to have anything to learn from — the same disagreement brevity restores for length, now restored for semantic robustness instead.

Status: implemented; the controlled measurement is in progress. Methods A–C and the reward term above are built in `latentsql/db/spurious.py` and `rewards.py`. The current experiment starts binary and spurious arms from the identical corrected two-role SFT checkpoint. Binary sets $w_{\text{atomic}} = w_{\text{memory}} = w_{\text{spurious}} = 0$; spurious sets them to 0.5, 0.3 and 0.5, respectively, so the spurious arm already includes dense partial credit for wrong rollouts and no separate dense arm is required for this comparison. Results are not inserted until both arms finish ordinary EX and the shared five-instance true-EX audit.

49 Latent rollouts

With `latent_rollouts: true`, GRPO first runs the same deterministic external retrieval used in SFT and evaluation, then samples SQL from the two-role latent prefix of Part V. It backpropagates through those latent steps, exactly as in curriculum SFT. There is no debate text to anneal away, so $w_{\text{fmt,start}} = w_{\text{fmt,end}} = 0$; both arms retain ordinary execution correctness, executable-SQL, tag-order and correct-only brevity terms, while only the spurious arm adds atomic, memory and stress-instance rewards.

50 What GRPO measurably did in the legacy six-role run

The following table is historical: fifty GRPO steps, fifty hard prompts, brevity enabled, starting from the original six-role stage-6 checkpoint and evaluated on the same held-out $n = 90$ mix. It demonstrates that the older latent implementation could learn from decoded-SQL variation, but it is not a result for the corrected two-role corpus, external-retrieval protocol or spurious reward.

	SFT (pre-GRPO)	after GRPO	Δ
EX ($n = 90$)	52.22%	**58.89%**	+6.67
bird EX	33.33%	**37.78%**	+4.45
spider EX	71.11%	**80.00%**	+8.89
avg output tokens	67.5	**61.9**	−5.6
unparseable	4	**0**	−4

Accuracy rose while output length fell — the brevity reward doing exactly what it was shaped to do — and every one of the 90 generations stayed parseable, so the latent → `<|consensus|>` → SQL format survived GRPO intact. The gain concentrates on Spider; GRPO's training prompts were drawn from the same length-filtered pool as the eval split, so the Spider number is the more independently-earned half of this result.

These legacy levers must not be added. GRPO's +6.67 began from a stage-6 checkpoint whose history contained the stage-4 collapse, whereas the repaired curriculum's +5.56 used a different lineage. The current controlled experiment does not attempt to repair that comparison; it starts both new reward arms from one newly trained two-role checkpoint.

Part XI
Does GRPO sharpen the agent subspaces, or not?

Part VI showed SFT builds separable per-agent directions. Part X added a second stage on top: GRPO. This Part asks the question those two leave open, and answers it the same way the rest of this document does: build it from one small fact, then check that fact against this project's own code and four papers read for this section (`pdfs/rl/`).

51 The one fact everything else follows from

Recall two things already proved earlier in this document.

Fact 1 (Definition 72). GRPO grades a *group* of G attempts at the same question and compares them to each other: $A_i = (r_i - \bar{r})/s$.

Fact 2 (Corollary 73). If all G attempts score the same, every $A_i = 0$, and the gradient is exactly zero. **No disagreement inside the group ⇒ nothing is learned.**

So the whole question "does GRPO reshape the latent thoughts?" reduces to one much smaller question:

When GRPO samples G attempts at one question, do the G latent thoughts actually differ from each other?

If yes, GRPO has something to compare and can push the latent thoughts apart or together. If no, it cannot — by Fact 2, with nothing to be zero exactly.

52 A die you roll versus a die you average

Here is the entire idea, before any formula. Take a fair six-sided die.

- **Roll it twice.** You might get 3, then 5. Two different numbers — *disagreement*, something to compare.
- **Compute its average twice.** $(1+2+3+4+5+6)/6 = 3.5$. Then do the same arithmetic again: 3.5. **Every single time, no exceptions.** An average has no randomness in it at all — it is one fixed number, not a draw from six possible ones.

Rolling produces disagreement. Averaging never does, no matter how many times you repeat it. This is not a subtle statistical point — it is just what the word "average" means.

Now recall equation (4.3), this project's own recipe for a continuous thought:

$$\text{align}_{\text{soft}}(h) \;=\; \sum_{v \in \mathcal{K}} \pi_v \, W_{\text{in}}[v, :].$$

Read the right-hand side out loud: it is a weighted *average* of 32 embedding rows. Not a roll. An average. Two independent facts confirm this is not a coincidence of wording:

1. **The identical formula appears, independently, in a 2026 GRPO paper.** Deng et al., *Latent-GRPO* (arXiv:2604.27998, their eq. 1), define a latent token exactly the same way:

$$s_t \;=\; \sum_{i=1}^{K} p_{t,i}\, e_{t,i},$$

a weighted average of the top-K embeddings by their softmax probability — the same construction as our (4.3), reached independently for the same reason (efficiency, no extra teacher forward pass). Two unrelated projects landing on the same formula is good evidence it is the natural default, not evidence that it is stochastic — it still is not.

2. **Nothing in `_latent_steps` or `align_soft` draws a random number.** Read the code (Part IV): h goes in, one fixed vector comes out. Run it on the same h a thousand times, get the same vector a thousand times.

This is the whole answer, stated plainly. Our continuous thought is the "average die" from the analogy above, not the "rolled die". Sample a GRPO group of G attempts at the same question with the same weights, and the G latent thoughts are the same computation on the same input, G times — so they come out identical, every time, for the same reason $3.5 = 3.5$.

52.1 Turning that into a one-line proof

Now state it as a proof, so "the same reason $3.5 = 3.5$" is not left as a slogan. Write $\Phi_k(\theta, x)$ for the latent prefix — the state after k role tags and their latent steps (Part V) — and write $\Phi_k(\theta, x)_i$ for the copy of it that rollout i of a GRPO group actually used.

Proposition 76 (the GRPO group shares one latent trajectory)**.** *For every rollout $i = 1, \ldots, G$ in one GRPO step, at one prompt x and one θ: $\Phi_k(\theta, x)_i = \Phi_k(\theta, x)$, identically. The only random variable that differs across i is the decoded completion y_i.*

Proof. Φ_k is a pure function of (θ, x) — no random draw anywhere inside it (§52, point 2). `grpo.py::_sample_group_latent` (l.236–299) computes it *once* and `_clone_caches` the result into all G rollouts (l.269–271); a clone of a fixed value is that same value, G times. `grpo.py::_latent_loss_inner` (l.331–369) instead *recomputes* it once per rollout via `build_prefix(...,grad=True)` (l.338–341) — but every rollout in one step shares the same θ (no `optim.step()` has run yet) and the same x, so a pure function fed the same input twice returns the same output twice: $3.5 = 3.5$, again. Either code path, Φ_k is constant in i; only y_i, built from each rollout's own `completion_ids`, is not. □

Corollary 77 (the latent-path gradient is blind to plan quality)**.** *The gradient of the latent-path loss with respect to the parameters that produce Φ_k can only be informed by comparing completions $y_1, \ldots, y_G$ conditioned on* one shared Φ_k *— never by comparing Φ_k against a different latent trajectory for the same x.*

Proof. By Proposition 76, Φ_k does not vary with i. The only i-dependent terms in the loss are the advantage A_i and $\log \pi_\theta(y_i \mid \cdot)$, both functions of y_i conditioned on the shared Φ_k; there is no term built from two different values of Φ_k for the gradient to use. □

53 Two published fixes, and their actual formulas

If a rolled die gives disagreement and an averaged die does not, the obvious fix is: **stop averaging, start rolling.** Two 2026 papers do exactly this, in two different places, and both prove their fix is mathematically sound rather than just plausible.

53.1 Fix 1 — roll inside the formula (Soft-GRPO / Latent-GRPO)

Deng et al. (arXiv:2604.27998, building on Zheng and Lee's Soft-GRPO) add **Gumbel noise** ξ_i to the log-probabilities *before* the softmax that builds the mixture weights:

$$g'_{t,i} = \log p_{t,i} + \xi_i, \qquad \alpha_{t,i} = \frac{\exp(g'_{t,i}/\tau_g)}{\sum_{u=1}^{K} \exp(g'_{t,u}/\tau_g)}, \qquad s_t = \sum_{i=1}^{K} \alpha_{t,i}\, e_{t,i}. \tag{10.2}$$

Because ξ_i is freshly random for every rollout, $\alpha_{t,i}$ — and therefore s_t — now differs from rollout to rollout, even for the same h: a rolled die, not an averaged one. Crucially, this is not just noise bolted on for its own sake: the Gumbel perturbation has a known, exact log-density (their eq. 3),

$$\log \pi_\theta(s_t) = \sum_{i=1}^{K} \Big[-(g'_{t,i} - \log p_{t,i}) - \exp(-(g'_{t,i} - \log p_{t,i})) \Big],$$

which is exactly what a policy-gradient method needs — the same role $\log \pi_\theta(y_{t+1} \mid \cdots)$ plays for an ordinary sampled token (Part X). With this in hand, the latent step can be treated by the *same* clipped-PPO-ratio machinery Part X already derived for text tokens, just piecewise-defined across the latent segment and the decoded segment (their eq. 5). Latent-GRPO then adds three further, narrower fixes on top of this base construction — *Invalid Sample Advantage Masking* (drop truncated/off-manifold rollouts from the group statistics before normalising), *One-sided Noise Sampling* (only allow $\xi_i \geq 0$, so a positive-advantage rollout's probability mass is never accidentally pushed the wrong way), and *Optimal Correct-Path First-Token Selection* (avoid averaging together two different correct latent paths into one invalid blended state) — because unconstrained two-sided noise on its own is unstable in practice; the base idea, however, is just: **put a real random draw inside the formula that builds the latent token.**

53.2 Fix 2 — roll the weights instead of the output (Dropout-GRPO)

Jung (arXiv:2606.10184) takes the opposite route: instead of perturbing the *output* of each latent step, perturb the *weights* that compute it. Draw one Bernoulli dropout mask ξ per rollout and hold it fixed across all T latent recurrence steps of that rollout:

$$h_t = f_{\tilde{\theta}(\xi)}(h_{t-1}), \qquad t = 1, \ldots, T, \tag{10.3}$$

where $\tilde{\theta}(\xi)$ is the dropout-perturbed weights and f_θ is one transformer block. Because ξ is drawn afresh per rollout, h_T (and so the latent prefix) genuinely differs across the G rollouts of a group — again, a roll rather than an average. Jung then proves, rather than just argues, that this is the right thing to do:

Theorem (Jung 2026, Theorem 3, restated). With the mean-only advantage $A^{(k)} = r^{(k)} - \bar{r}$ and the mask replayed at update time,

$$\mathbb{E}\big[\nabla_\theta \mathcal{L}_{\text{surr}}(\theta)\big]_{\theta=\theta_{\text{old}}} = \Big(1 - \tfrac{1}{K}\Big)\, \nabla_\theta J(\theta)\Big|_{\theta_{\text{old}}},$$

where $J(\theta)$ is the expected reward of the *mask-averaged* policy. The surrogate gradient is an unbiased estimate of the true gradient, up to a known, constant scale factor $(1 - 1/K)$ — not an approximation with unknown error.

On GSM8K they report a Coconut baseline improving from 27.29% to 29.01% pass@1 once this fix is applied — direct evidence that giving the latent step somewhere to disagree turns GRPO from stalled into working, on a Coconut-style model closely related in spirit to this project's own latent mechanism.

54 Neither fix is in this project's code

Neither (10.2) nor (10.3) appears anywhere in `latentsql/`. The latent step (Part IV) is exactly $\text{align}_{\text{soft}}(h)$, computed once, with no injected randomness of any kind. So the averaged-die picture of §52 is the literal, exact situation here, not an analogy that only approximately applies — confirming Proposition 76 and Corollary 77 a second, independent way: not just by reading the sampling code, but by checking the mechanism against the two published recipes for the alternative and finding neither one present.

In one sentence: whatever this project's GRPO run changed, it did not change it by comparing one plan against a different plan for the same question — because every rollout in every group was handed the exact same plan.

55 So why did our own run still move the numbers?

Part X measured +6.67 EX from GRPO. If the latent step never disagrees within a group, where did that gradient come from?

From the one place that still rolls: the SQL decode. After the shared latent prefix, this project's own `_sample_group_latent` (l.276–291) samples the actual SQL tokens with `torch.multinomial` at `temperature=1.0` — a real die roll, every rollout, every token. Two rollouts starting from the *identical* latent state can still decode to two *different* queries, one correct and one not, and Fact 2 (§52.1) needs exactly that kind of disagreement to produce a nonzero gradient. So the group's advantage was not forced to zero the way Jung's Coconut baseline was — but the resulting gradient could only ever be shaped like "write better SQL from this plan", never "pick a better plan." This is a plausible mechanism consistent with the measured numbers, not a controlled test of it: no run at `temperature=0` has been made for comparison.

55.1 How SQL reward still changes the shared latent trajectory

"Shared" does not mean "frozen." It means that every rollout sees the same latent value before the update. During the update, `_latent_loss_inner` rebuilds that value with `build_prefix(..., grad=True)`. Autograd can therefore follow the SQL loss backward through the whole latent computation.

Write the deterministic latent prefix as $z_\theta = \Phi_\theta(x)$. The probability of a sampled SQL query is then a composition of two parameterised maps:

$$p_\theta(y \mid x) = \pi_\theta(y \mid x, z_\theta) = \pi_\theta(y \mid x, \Phi_\theta(x)) .$$

For a group of sampled queries, the implemented latent-path loss is:

$$\mathcal{L}_{\text{latent}}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{i=1}^{G} A_i \log \pi_\theta(y_i \mid x, \Phi_\theta(x)) . \tag{10.4}$$

The execution engine is not differentiable. GRPO does not differentiate through it. The engine only produces the scalar advantage A_i. The differentiable object is the log probability of the sampled SQL. Applying the chain rule gives two gradient paths:

$$\nabla_\theta \log \pi_\theta(y_i \mid x, \Phi_\theta(x)) = \left.\frac{\partial \log \pi_\theta(y_i \mid x, z)}{\partial \theta}\right|_{z=\Phi_\theta(x)} + \nabla_z \log \pi_\theta(y_i \mid x, z)\big|_{z=\Phi_\theta(x)}^{\top} \nabla_\theta \Phi_\theta(x). \tag{10.5}$$

The first term changes the SQL decoder directly. The second term changes the parameters that created the latent prefix. A positive advantage moves those parameters toward a state that makes the rewarded SQL more likely. A negative advantage moves them away from a state that supports the penalised SQL. The stored prefix from the completed rollout does not change. The same question produces a new prefix after the parameter update.

Proposition 78 (zero-mean advantages need not cancel the latent gradient)**.** *Suppose two SQL rollouts share one latent prefix z. Let their advantages be $A_+ = a$ and $A_- = -a$, where $a > 0$. The gradient with respect to the shared prefix is:*

$$\nabla_z \mathcal{L}_{\text{latent}} = -\frac{a}{N_{\text{tok}}} \left(\nabla_z \log \pi_\theta(y_+ \mid x, z) - \nabla_z \log \pi_\theta(y_- \mid x, z)\right) . \tag{10.6}$$

It is non-zero whenever the two SQL queries have different score gradients at z.

Proof. The two advantages sum to zero. Their multipliers cancel only when the vectors they multiply are equal. Different SQL sequences generally have different token log probabilities and different score gradients. Substituting $A_+ = a$ and $A_- = -a$ into (10.4) gives (10.6). Therefore zero mean reward advantage does not imply zero latent-state gradient. □

This proves the limited claim precisely. Current GRPO can reshape one deterministic latent computation so that it supports rewarded SQL more strongly. It still cannot learn that one latent plan is better than another for the same question because the group contains no second latent plan.

56 A separate question: did SFT even need to *create* the subspaces?

One more complication, about Part VI rather than about GRPO. IMAD's own number: base-model steering AUC = 0.802, not ≈ 0.5 (chance). That is already well above chance *before* any internalisation. Two 2026 papers, read for this section, suggest why that might be:

- **Persona Vectors** (Chen, Arditi et al., arXiv:2507.21509) extract trait directions by the exact same difference-in-means recipe as Definition 48, and find that a model's finetuning-induced *behaviour* shift on a trait correlates strongly with its finetuning-induced *activation* shift along that trait's own vector ($r = 0.76$–0.97 across traits tested) — finetuning genuinely moves activations along directions a linear probe already finds.

- **Tracing Persona Vectors Through Pretraining** (Moskvoretskii et al., arXiv:2605.13329) trace *when* such directions first appear, and find they form *"within* 0.22% *of OLMo-3 pretraining"* — almost the very start — and remain usable through the entire post-trained model.

The honest, revised claim. "Internalisation creates agent subspaces from nothing" overstates it. "Internalisation makes directions — which may already exist from pretraining — more reliably separable and steerable" is what 0.802 → 0.903 and the two papers above jointly support. Theorem 51 is unaffected either way: difference-in-means is still the optimal linear probe for whatever separation exists, whenever in training it first appeared.

57 Summary

Proven here	Cited, correctly scoped	Open
The GRPO group shares one latent trajectory — proved by reading `grpo.py` (Proposition 76) and by checking that neither published randomness-fix is present (§54).	Two published, proved fixes exist: Gumbel noise in the formula (Latent-GRPO, eq. 10.2), or a shared dropout mask in the weights (Dropout-GRPO, eq. 10.3, exact $(1 - 1/K)$ unbiasedness).	Whether `05_steering.py` run before/after `03_grpo.py` would show the per-role subspaces sharpening, unchanged, or degrading.
By Fact 2, this means the latent-path gradient can never compare two plans for the same question — only two completions of the same plan.	IMAD's base-model AUC (0.802) is high enough to be consistent with pretraining-era structure the persona-tracing paper documents.	Whether applying either published fix to this project's own latent path would change training stability or the resulting geometry.

58 A sibling project, and a genuinely different fix to a shared problem: SERL-SQL

One more comparison is worth making precisely, against a 2026 Text-to-SQL GRPO paper read specifically for this section: **SERL-SQL** (Liu, Feng, Li et al., arXiv:2608.00485, "Selective Hindsight Distillation for Text-to-SQL Reinforcement Agentic Learning"). It targets the exact coarseness this document's own GRPO derivation has (Definition 72), but fixes it a completely different way, in a completely different architecture.

58.1 The same coarse-credit problem, independently named

Recall our own advantage (Definition 72): one scalar A_i per rollout, and recall from (9.1)/§X that this single number is *broadcast identically* onto every token of that rollout's loss (`per_token = pg + kl_beta*kl`, with `adv` the same A_i at every position). SERL-SQL names this exact structure as its starting complaint, in their own notation (A^n their per-rollout advantage, m, n indexing token/trajectory):

> *"Although this objective provides a stable reward-aligned update direction, its credit assignment remains coarse: decisive SQL actions, reasoning tokens, and formatting tokens*

receive the same trajectory-level advantage."
"In standard trajectory-level GRPO, this reward-derived advantage is assigned to every action step in the trajectory, i.e. $A_t = A^n$."

That second sentence is, symbol for symbol, exactly what our own `adv` broadcast does. Two independent projects, applying GRPO to the same task (Text-to-SQL), diagnosed the identical structural fact.

58.2 Two different fixes to the same complaint

This document's own fix (Part XI) targets a *different* symptom of trajectory-level advantage: that every rollout in a group also shares one *latent* prefix (Proposition 76), so the gradient cannot compare two plans, only two completions of one plan. SERL-SQL's architecture has no latent prefix at all — everything is decoded text — so it instead targets the *token-level* symptom directly: give different tokens *within one already-decoded trajectory* different credit.

Their mechanism: a second, *teacher* policy π_T re-scores the student's own tokens *after* seeing execution feedback the student did not have at rollout time (the actual query result, or the actual error message) — information available only in hindsight. The gap between teacher and student log-probability at each token,

$$\Delta_{t,i} = \log \pi_T(\cdot) - \log \pi_\theta(\cdot),$$

is converted to a bounded, sign-aware reweighting factor

$$w_{t,i} = \mathrm{clip}\big(\exp(\mathrm{sgn}(A^n)\,\mathrm{sg}[\Delta_{t,i}]),\ w_{\min},\ w_{\max}\big),$$

applied *only* to SQL/tool-action tokens via a mask $m_{t,i}$ (formatting and reasoning tokens keep weight 1):

$$\bar{w}_{t,i} = (1 - m_{t,i}) + m_{t,i}\, w_{t,i}, \qquad \widetilde{A}_{t,i} = A^n\big[(1-\alpha_k) + \alpha_k\, \bar{w}_{t,i}\big],$$

with α_k decaying so the teacher's influence fades over training. The sign-preserving clip keeps the update direction fixed by A^n while the *magnitude and locality* of credit is reshaped by hindsight — a genuine per-token fix, orthogonal to ours.

Neither fix solves the other's problem. SERL-SQL's token reweighting cannot address Proposition 76, because it has no shared latent prefix to begin with. Our fix (a lower late-stage learning rate, Part IX) says nothing about *which token* inside a decoded SQL query deserves more credit than another — it never distinguishes tokens within a rollout at all. The two problems just happen to both trace back to the same root cause named in Definition 72: one scalar advantage per rollout, applied uniformly wherever it is used.

58.3 A speculative bridge (unimplemented, stated as such)

SERL-SQL's core idea — score a token using information unavailable at rollout time, then convert that into a bounded reweighting factor — is architecture-agnostic in principle. One could imagine an analogous *hindsight latent critic*: after decoding the final SQL and learning whether it executed correctly, re-score the two latent role blocks (Part V) for how much each contributed, and reweight Corollary 77's shared-prefix gradient unevenly across their four latent positions. **This is not built or tested** — it is recorded only as a possible response if the open question about GRPO reshaping latent geometry requires one.

58.4 The one architectural difference that is not open to interpretation

Beyond credit assignment, SERL-SQL and LatentSQL make opposite bets about *where reasoning lives at inference time*, and this part is not speculative — it follows directly from how each system is described.

	SERL-SQL	**LatentSQL (ours)**
Where does reasoning happen	explicit text, every turn decoded and readable	latent vectors, zero tokens decoded per role (Part IV)
Does inference re-execute against the database	yes — the deployed policy runs its own "think, act, and observe" loop each turn, querying the database live	no — SFT uses offline labelled one-edit failures to teach error anticipation, and GRPO uses execution only as a training reward; nothing is executed before the final SQL is produced
Number of decoding turns per question	multi-turn (generate, execute, revise, repeat)	one (a single forward pass through two latent role blocks, then decode)
Model scale reported	7B / 14B / 32B	7B backbone (student), no 32B configuration run

SERL-SQL keeps the execution-feedback loop *live*: every deployed question pays for database round-trips and decoded think/act/observe turns. LatentSQL makes the narrower bet that offline failure-labelled examples can teach the SLM to *anticipate* common execution failures and that GRPO can reinforce final-query behaviour, without claiming that weights reproduce the information content of a fresh runtime error. The current two-role experiment is needed before comparing accuracy; the older 57.78% mixed-set number belongs to the legacy six-role checkpoint and is not evidence for the revised method.

Part XII

Complete map from equation to code

Equation	What it is	Where it lives
(2.1)–(2.2)	transformer, LM head, softmax	standard HF model internals
(4.1)	latent-mode input substitution	`latent.py::_latent_steps`, l.219–244
(4.2)	linear alignment $W_a = W_{\text{out}}^{\top} W_{\text{in}}$	`latent.py::build_aligner`, `mode="linear"`
(4.3)	soft-embedding alignment	`latent.py::build_aligner`, `align_soft`
$\mathcal{C}_t$ cache carry	`share_working_memory`	`latent.py::build_prefix`, l.509–535
crop no-op/undo	checkpoint-safe cache mutation	`latent.py:220,359`
(5.1)–(5.2)	two-role loop and stage k	`latent_sft.py::_coconut_sequence_loss`
explicit $T_{k+1}, \ldots, T_2$ suffix	Coconut curriculum supervision before each role becomes latent	`latent_sft.py::_coconut_sequence_loss`
(external v)	shared deterministic value retrieval	`value_index.py::retrieve_value_hints`
training-only failure labels	one-edit mutants and offline execution outcomes	`latent_sft.py::_offline_execution_negatives`
(7.1)–(7.2)	masked cross-entropy, main loss	`latent_sft.py`, final `model(...,labels=...)` call, l.157–163
(7.3)–(7.4)	SQL-readiness probe	`latent_sft.py:130--150`
Proposition 60	`idx+1==sql_ready_role`	`latent_sft.py:130`
`sql_ready_active` gate	$k \geq j^{\star}$ requirement	`latent_sft.py`, l.500–506
(8.1)	LoRA	`configs/*.yaml`: `sft.lora_r=32,lora_alpha=64`
(8.2)	Adam	`latent_sft.py:407`, `torch.optim.AdamW(params,lr=cfg.lr)`
grad-accum = batch size	Example 65	`run_chess_coconut_sql_direct.sh`, `--grad-accum"$limit"`
optimiser reset at stage boundary	moment wipe	`run_chess_coconut_sql_direct.sh`, `rm-f"$OPTIM_STATE"`
`clip_grad_norm_(params,1.0)`	gradient clipping	`latent_sft.py:597`
`gpu_settle`	CUDA-context-release wait	`run_chess_coconut_sql_direct.sh`
`LATENTSQL_REQUIRE_CUDA`	silent-CPU-fallback guard	`hf_local.py::resolve_device`
(9.1)	GRPO group advantage	`rewards.py::group_advantages`
clipped surrogate; latent KL disabled in the current run	PPO-style GRPO loss with $\beta_{\text{KL}} = 0$	`grpo.py` and the active config
(9.2)	IMAD reward eq. (1)	`rewards.py::DebateReward.__call__`
w_{fmt}, l anneal	`RewardSchedule`	`rewards.py::anneal,RewardSchedule`
dense partial credit	atomic/memory terms	`rewards.py`, l.235–251
brevity term	correct-only tie-breaker	`rewards.py`, l.224–233
spurious penalty	deterministic stress-instance agreement	`db/spurious.py`, `rewards.py`
(9.3)–(9.4)		
(6.1)/IMAD eq.(3)	difference-in-means	`caa.py`, steering-vector extraction
(6.2)	activation addition	`caa.py::steer_layer`
Definition 1	EX and ordered/unordered match	`latentsql/db/execute.py`
Part XI	GRPO's effect on agent subspaces	`05_steering.py--compare-base`, run before/after `03_grpo.py`

59 The corrected end-to-end pipeline

1. `01_build_traces.py` runs the teacher system and stores its traces.
2. `02d_prepare_latent_records.py` selects round-0 CHESS candidates with EX = 1, runs shared external value retrieval, and creates the two-role records with offline mutant outcomes.
3. `02c_latent_sft.py` applies the two-stage Coconut curriculum: explicit candidate trace at stage 1, then two fully latent roles at stage 2.
4. `03_grpo.py` starts binary and spurious+dense arms from the identical stage-2 checkpoint, using external retrieval in both.
5. `04_eval.py` reports ordinary held-out EX with the same retrieval path; `05_true_ex.py` then reports robust true EX and the percentage of ordinary-correct predictions exposed as spurious.
6. `07_probe_latent.py` can save top vocabulary projections, an SQL decode stopped after each role, referenced tables and columns, execution outcomes, changes from the previous stage and an EOT verbalisation. These are probes of what the state can verbalise, not exact decodings of the hidden computation.

The complete controlled run is orchestrated by `scripts/run_two_role_binary_spurious.sh`; `scripts/run_chess_coconut_sql_direct.sh` drives its two curriculum stages.

Part XIII
Results, reported honestly, including where the method loses

60 The two tests that actually define success

Every number below answers one of two questions, and they must not be conflated.

Test A (internalisation). Does the internalised student reach the same accuracy as the SFT student it started from, while its *output tokens* collapse? Reference: the SFT checkpoint. This is a test of compression.

Test B (the thesis). Does the internalised student match or beat the explicit multi-agent system *at the same model size*, at a fraction of the tokens? Reference: the explicit pipeline on the same backbone — not the larger teacher that generated the training traces.

What is explicitly not the target. A student beating the 7B teacher that generated its training data is not the claim: the teacher is a ceiling on trace quality, not a bar to clear. Recorded teacher EX on an early panel run was 64.2%; a 3B student scoring below that is not a failure of this method.

60.1 An earlier, text-mode arm: Test A never passed

Before the legacy six-role SQL-direct curriculum, an earlier arm trained IMAD's original text-mode internalisation (write the full debate, then anneal it away by length-clipping). Across three configurations of that arm, generated length moved 1024 → 922 tokens — essentially flat. Three causes were diagnosed and are recorded for completeness:

1. the length clip l was annealed *below* the natural trace length from the first step, so the pressure to shorten was present from step 1 with no headroom to explore first;
2. `max_new_tokens` sat below l_{start}, so 16 of 24 logged steps had every rollout truncated to an identical length — nothing left to reward for being shorter (a direct instance of Corollary 73);
3. only 24 GRPO steps were run; IMAD-scale internalisation used hundreds.

This motivated replacing inference-time verbalisation structurally. The current curriculum improves on the legacy SQL-direct variant: roles not yet internalised remain explicit and supervised, and only after their content has trained the student are they replaced by latent states. At inference both roles are latent and only SQL is decoded.

61 Test B on the earlier arm, before the architectural change

reference	EX	output tokens
CHESS @ 3B (the bar)	48.89%	1009
ours, SFT-only @ 3B	52.22%	1242

Half-passed: accuracy clears the bar (+3.3 points), tokens do not (1242 vs. 1009) — and the token axis is the entire point of the method, so this arm did not establish the thesis on its own.

62 Legacy six-role SQL-direct results

These measurements come from the superseded six-role architecture on the 7B backbone and an $n = 90$ held-out BIRD+Spider mix. They are retained as development history and evidence that latent decoding can compress output, but they do not measure the two-role architecture derived in Parts V–X.

run	EX	bird	spider	avg out tokens	unparseable
Previous full-text SFT student	56.67%	31.11%	82.22%	1339.3	–
Curriculum, stage 6 (original)	52.22%	33.33%	71.11%	67.5	4
Curriculum, stage 6, **retrained**	**57.78%**	**37.78%**	**77.78%**	**61.2**	**0**
Curriculum, stage 6 + GRPO (from original)	**58.89%**	37.78%	**80.00%**	61.9	0

Within that legacy experiment, output tokens fell 95% (1339.3 → 61.2–67.5) while the best EX exceeded the full-text SFT reference. This justifies continuing the latent approach; it cannot be transferred numerically to the revised corpus or reward comparison.

63 Current two-role binary-versus-spurious experiment

The current experiment uses 462 validated records. Every record has exactly two role turns and ends in a correct first-pass CHESS candidate labelled EX = 1; 452/462 records contain at least one offline negative outcome. The two-stage latent SFT runs once over this shared corpus. Its final checkpoint is then copied logically into two GRPO arms trained on the same 192 examples: binary removes atomic, memory and spurious terms, while spurious retains atomic and memory partial credit and adds stress-instance agreement. Both are evaluated on the same 90 examples with ordinary EX and then on up to five deterministic stress databases per example with true EX. No result is reported here while the run is incomplete.

arm	SFT start	wrong-case dense terms	spurious term	reported metrics
binary	shared two-role checkpoint	no	no	ordinary EX, true EX
spurious	shared two-role checkpoint	atomic + memory	yes	ordinary EX, true EX

64 Cost

	greedy, latent path	explicit multi-agent (measured elsewhere in this doc)
Output tokens/question	61–68	≈ 1339
Latency/sample	2.07–2.21 s	(multiple sequential LLM calls)
Peak GPU	15.9 GB	–

64.1 Speculative decoding: measured, and a dead end on this path

Speculative decoding drafts several tokens with a small model and verifies them in one pass of the target, provably preserving the target's exact output distribution at greedy temperature (verification is lossless). It was measured at three draft sizes on the retrained-lineage GRPO checkpoint:

draft	EX	latency/sample	total	vs. greedy	acceptance length
none (greedy)	58.89%	2.09 s	188 s	1.00×	–
Coder-0.5B bf16	57.78%	2.99 s	269 s	0.70×	2.26
Coder-3B bf16	57.78%	3.39 s	305 s	0.62×	2.37
Coder-7B 4bit	57.78%	4.26 s	383 s	0.49×	2.34

Speculation is slower at every draft size measured, and the reason is structural rather than a matter of tuning: acceptance length stays flat (2.26–2.37) across a 14× range of draft size, while wall-clock improves only 1.42× over that same range — fixed per-round overhead dominates, not draft quality. The draft model cannot see the target's latent prefix (it conditions on the plain-text prompt), so it must re-prefill the whole ≈ 1570-token prompt to cover only ≈ 62 generated tokens — speculation needs long generations to amortise its overhead, and this architecture's entire point is emitting few tokens. **Practical guidance: use greedy decoding for the latent path.**

65 Smoke-test coverage (verifies mechanism, not accuracy)

Two tiers, run CPU-only, no GPU, no data, no API key. Tier 1 (standard library only) builds two SQLite databases, runs the teacher and all four baselines, and checks trace format and every reward term. Tier 2 (needs torch) loads a tiny model and runs one real step of SFT, one GRPO step with a genuine execution-based reward, latent generation through all three aligners (Section 16), latent working memory, and steering-vector extraction and hooking. All 28 smoke checks and all 25 unit tests pass.

> Tier 2 verifies shapes, masks, caches, hooks, and schedules — *not* model quality. A tiny model cannot write correct SQL; the point is only that nothing in the mechanism is broken before a GPU-week is spent on it.

Part XIV
What is not established

Honesty about limits is part of the mathematics of this document, not an afterthought, so it gets its own Part rather than a footnote.

66 Required ablations, not yet run

Ablation	How to run it	The objection it answers
Full trace vs. final answer only	`02_sft.py --consensus-only`	is the debate transcript doing anything, or only the final answers (DebateGPT, IMAD Table 1)?
Debate panel vs. one structured format	`mas.agents: [query_plan]`	is a single execution-plan chain-of-thought already enough?
GRPO from base, no teacher	`03_grpo.py --from-examples bird --sft /nonexistent`	is the multi-agent teacher earning its cost over a pure execution-reward RL run?

The third is uncomfortable and must be reported even if unflattering: published execution-reward-only pipelines reach 68.9% BIRD dev (Arctic-Text2SQL-R1-7B) and 67.1% (SLM-SQL-1.5B) without any teacher at all. The honest external bar for this project is those numbers, not the explicit CHESS baseline.

67 The historical attribution gap in the stage-4 repair

In the legacy six-role retrain, two knobs (learning rate and probe role) moved together. The measured $\mathcal{L}^{\text{probe}} \sim 10^{-4}$ makes the learning-rate explanation more likely, but the clean legacy ablation was not run. This gap does not contaminate the current experiment: its SQL-readiness weight is explicitly zero and its two stages use the same declared learning rate.

68 What Part VI does not establish for this system

Theorem 51 and IMAD's steering result establish that internalised multi-agent debate *can* produce separable per-agent subspaces in IMAD's setting. Whether this project's schema-linker and candidate-generator blocks occupy separable subspaces has not been measured. The extraction code exists and applies unchanged, but the measurement must be run on the corrected two-role checkpoint.

69 What Part XI does not establish

Proposition 76 and Corollary 77 prove a structural limit on what the latent-path GRPO gradient can compare within one group; they do not show what happens to role-subspace geometry across training. Answering that requires probing the shared pre-GRPO two-role checkpoint and each

post-GRPO arm. Separately, whether the two role blocks are linearly separable before RL is itself unmeasured.

70 What Part III's capacity argument does not establish

Restated from Section 13.2: the DPI argument bounds what a separate-call pipeline can transmit and shows that the four-position latent channel has a much higher storage ceiling than the measured transcript. It does not show that training fills this ceiling with useful reasoning rather than near-duplicate states. That question is empirical and belongs to subspace and per-role diagnostic probes.

71 What Part X does not establish

The legacy GRPO gain and legacy curriculum-repair gain came from different six-role starting checkpoints and must not be summed. More importantly, neither answers the current question. The controlled two-role run fixes the starting checkpoint and changes only the reward arm, then reports both ordinary EX and true EX; conclusions must wait for that run to finish.

72 Infrastructure and scale limits

- The teacher's own quality bounds the student's: with a local 7B teacher, traces are weaker than a GPT-4-class teacher would produce. The execution-based filter discards wrong traces but cannot manufacture correct ones.
- GRPO rollouts are the throughput bottleneck – group_size $\times$ prompts generations per step, each executed against a real database; the `ExecutionOracle` cache is what makes this tractable at all.
- `latent` mode decodes token-by-token in Python with no KV-cache fast path through `generate()`, so it is slower per decoded token than plain text mode even though it emits far fewer tokens overall.
- The mock LLM backend's numbers are plumbing checks only, never cited as results.
- Completed legacy results use one fixed $n = 90$ held-out mix and rolling $n = 30$ probes. The current protocol retains the $n = 30$ probes only as smoke checks and reserves comparison for the shared $n = 90$ ordinary-EX and true-EX evaluation.

Part XV

Every formula in one place

Level 0 — the model as a function of vectors

$$H_t = \text{Transformer}(E_t), \qquad h_t = H_t[t,:], \qquad p(x_{t+1} \mid x_{\le t}) = \text{softmax}(W_{\text{out}} h_t)$$

Level 1 — the decode bottleneck

$$I(H;T) \ge I(H;H') \quad \text{(DPI)}, \qquad I(H;T) \le L \log_2 |V| \quad \text{(ceiling)}$$

Level 1b — latent search over partial SQL derivations

$$F_{c+1} = F_c \cup \{s' : \exists s \in F_c, s \to s'\}, \qquad z_c = |F_c|^{-1/2} \sum_{s \in F_c} u_s$$

$$\pi_r(z \mid x) = \frac{\pi_{\text{ref}}(z \mid x) e^{r(x,z)/\beta}}{\sum_{z'} \pi_{\text{ref}}(z' \mid x) e^{r(x,z')/\beta}}, \qquad p_\beta = \frac{p e^{1/\beta}}{p e^{1/\beta} + 1 - p} > p$$

Level 2 — the continuous thought

$$\text{latent mode: next input} = \text{align}(h_t), \qquad \text{align}_{\text{soft}}(h) = \sum_{v \in \text{top}_k(W_{\text{out}} h)} \pi_v W_{\text{in}}[v,:]$$

Level 3 — the role chain and curriculum

$$\text{stage } k: \quad c \,\|\, \text{tag}(r_1) \bullet^m \,\|\, \cdots \,\|\, \text{tag}(r_k) \bullet^m \,\|\, T_{k+1:2} \,\|\, \texttt{<|consensus|>} \,\|\, y^\star$$

Level 4 — difference-in-means

$$d = \mu_+ - \mu_-, \qquad \hat{d} = d/\|d\|, \qquad w^\star = \arg\max_{\|w\|=1} J(w) = \hat{d} \text{ (under isotropic noise)}$$

Level 5 — the training loss

$$\mathcal{L}_k^{\text{main}} = -\frac{1}{|U_k|} \sum_{i=1}^{|U_k|} \log p_\theta(u_i^{(k)} \mid \Phi_k, u_{<i}^{(k)}), \qquad \mathcal{L}_k = \mathcal{L}_k^{\text{main}} \quad (\lambda_{\text{probe}} = 0 \text{ in the current run})$$

Level 6 — the optimiser

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \qquad \text{step 1 after reset: } \Delta\theta \approx -\eta \,\text{sign}(g)$$

Level 7 — LoRA

$$W = W_0 + \frac{\alpha}{r} BA$$

Level 8 — the RL objective

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}\big[\nabla_\theta \log \pi_\theta(y \mid x) R(x,y)\big], \qquad A_i = \frac{r_i - \bar{r}}{s}$$

Level 9 — the reward

$$r(x,y) = R_{\text{EX}} + w_{\text{exec}} R_{\text{exec}} + w_{\text{brev}} R_{\text{brev}} + w_{\text{atomic}} R_{\text{atomic}} + w_{\text{memory}} R_{\text{memory}} - w_{\text{spurious}}(1 - R_{\text{spurious}})$$

Level 10 — the metric

$$\text{EX}(y, y^\star, D) = \mathbb{1}\big[\text{exec}(y, D) = \text{exec}(y^\star, D)\big]$$

$$\text{TrueEX}(y) = \text{EX}(y, y^\star, D_0) \prod_{D \in \mathcal{I}(y^\star)} \mathbb{1}\big[\text{exec}(y, D) = \text{exec}(y^\star, D)\big]$$

73 The one-paragraph version

LatentSQL keeps database-dependent work honest: deterministic value retrieval remains external and identical at training and inference, while a single SLM internalises only schema linking and candidate generation through four continuous positions before decoding SQL. The two-stage Coconut curriculum first supervises the explicit candidate trace, including deterministic offline one-edit failures, and then replaces that trace with latent states; this supports the claim that the model is trained to anticipate execution failures, not that it receives runtime feedback. The final shared SFT checkpoint feeds two controlled GRPO arms: binary and spurious, with the latter already containing dense wrong-answer rewards. Ordinary EX is reported separately from true EX on deterministic stress databases. Earlier six-role results and their stage-4 collapse remain useful development history, but they are not evidence for the revised two-role method; the present claims must follow the current evaluation rather than inherit old numbers.